

MDR Advocacia

— Tecnologia da Informação —

# DunaFlow

by DUNATECH

## Plataforma de Orquestração Jurídico-Operacional

*Automação de tarefas, inteligência de publicações e monitoramento processual  
integrados ao Legal One*

### **Documento de Projeto — visão executiva e técnica**

Versão 1.6 — Abril/2026 · documento vivo

*Preparado pela equipe de TI — MDR Advocacia*

## Sumário

## Sumário Executivo

O **DunaFlow** é a plataforma proprietária da MDR Advocacia que reúne, em um único sistema, a orquestração de tarefas, a inteligência sobre publicações judiciais e o monitoramento de carteiras processuais. Ele foi concebido para reduzir o trabalho manual repetitivo, aumentar a consistência nas agendas e trazer visibilidade operacional em tempo real para as equipes jurídicas e de back-office.

Construído sobre um stack moderno (FastAPI, PostgreSQL, React e SQLAlchemy) e integrado nativamente ao Legal One, o DunaFlow transforma rotinas de horas em operações de minutos: o que antes dependia de trabalho cadastro-a-cadastro no CRM hoje acontece de forma automatizada, rastreável e auditável. A plataforma combina agendamento em lote (via planilhas e APIs), classificação inteligente de publicações com modelos de IA, tratamento automatizado no Legal One (RPA), monitoramento processual com bases públicas (DataJud e DJEN/Comunica) e automações agendadas que rodam em background.

### Principais conquistas até o momento

- Agendamento em lote unificado: três estratégias (Planilha, Onesid e Onerequest) convergem para um mesmo fluxo de criação de tarefas no Legal One, com dashboard próprio de acompanhamento.
- Classificação de publicações com IA: pipeline baseado em Claude Haiku 4.5 com custo estimado de US\$ 0,0017–0,003 por publicação no Batch API — praticamente gratuito em escala.
- Retentativas inteligentes com backoff exponencial para respeitar limites de taxa das APIs externas e manter a resiliência do sistema em cargas altas.
- Reclassificação manual instantânea, preservando decisões humanas e evitando novas chamadas ao Legal One (driver principal do custo e do rate-limit).
- Monitoramento processual multifontes: tabelas analíticas para DataJud + DJEN/Comunica, régua de maturidade em 7 níveis e fila operacional para validação de baixa.
- Automação de tratamento de publicações no Legal One via RPA Playwright, com verificação explícita de estado e prevenção de travamentos.
- Automações agendadas via APScheduler: rotinas recorrentes (ex.: "diário 07h — puxar e classificar SP") rodando sem intervenção.
- Administração completa: gestão de usuários, setores, squads, permissões e modelos de tarefas, com autenticação por JWT.

### Retorno esperado

O DunaFlow ataca simultaneamente custo, velocidade e qualidade. Reduz horas de trabalho humano em cadastros repetitivos, diminui o risco de erros em agendamentos críticos (audiências, prazos, cumprimentos), concentra a gestão em dashboards únicos e oferece uma camada de inteligência —

classificação + monitoramento — que antes simplesmente não existia. O resultado é uma operação jurídica mais previsível, auditável e escalável.

#### Em uma frase

*O DunaFlow transforma o Legal One de "sistema de registro" em "sistema de operação automatizada", com rastreabilidade, inteligência artificial aplicada ao jurídico e integração com as principais fontes públicas de dados judiciais do Brasil.*

### Sobre esta versão

Este é um documento vivo — cada versão consolida as evoluções do produto desde a versão anterior. A versão 1.0 (abril/2026) descreveu a plataforma em seu primeiro release consolidado. A versão 1.1 incorporou o primeiro grande ciclo de melhorias pós-v1.0, com foco em desempenho para equipe, feedback loop do classificador, tratamento de publicações obsoletas e robustez contra rate-limits. A versão 1.2 trouxe um ajuste fino na experiência de triagem de publicações — novo filtro por natureza, reorganização da barra de filtros e visibilidade do CNJ em registros ainda não vinculados. A versão 1.3 ampliou o módulo de Publicações com filtro por polo, link direto ao Legal One no modal, extração de CNJ a partir do texto, retry individual de falhas de tratamento e correção do classificador para arrays truncados. A versão 1.4 consolidou o rebrand da plataforma para DunaFlow (by DUNATECH), reconstruiu a Home como um dashboard executivo com KPIs, funil e velocidade, adicionou reaproveitamento de syncs órfãs, leitura de overrides de classificação no grid de cobertura e ajustes de responsividade para uso em celular. A versão 1.5 estreou o módulo de Agendar Prazos Processuais (intake de PI + classificação IA + HITL + export), blindou o agendamento de tarefas com checagem prévia de duplicatas no Legal One, estendeu multiseleção a todos os sete filtros de Publicações, evoluiu a administração de Templates (responsável opcional, paginação, multiseleção de escritórios, cobertura com collapse) e estabilizou a integração Thomson Reuters no Tratamento Web (login em dois passos, OnePass SSO, auth handoff). A versão 1.6 — a atual — é o maior salto de escopo do produto até aqui: nascem quatro módulos novos ao mesmo tempo. OneRequest orquestra as DMIs do Banco do Brasil de ponta a ponta (intake, tratamento, agendamento no L1, alerta Teams, dashboard próprio). Minha Equipe traz observabilidade de performance por equipe, com painel de vazão, raio-x individual, relatórios PDF via Sonnet, ingestão automática do Agenda Analytics do Legal One e cancelamento em massa de tarefas duplicadas. Balanceador de Agenda entrega redistribuição AO VIVO de carga entre colaboradores do L1, com reatribuição real de responsáveis via API. Análise Recursal, dentro do módulo renomeado Prazos Processuais, faz upload de PDFs de processo, gera veredito de viabilidade recursal e parecer formatado via Sonnet, com custas dos 27 TJs, prazo fatal determinístico e OCR direcionado para documentos-imagem. O capítulo "Parte VI — Histórico de Atualizações" detalha cada mudança.

## Parte I — Visão de Negócio

### 1. Contexto: MDR Advocacia e os desafios de advogar em escala

A MDR Advocacia atua com operações judiciais massivas: dezenas de milhares de processos, publicações diárias em múltiplos tribunais, audiências, cumprimentos, embargos, despachos — cada um deles exigindo tipificação correta, agendamento preciso e acompanhamento até a conclusão. Nesse cenário, o Legal One cumpre o papel de CRM jurídico, mas por si só não resolve três dores estruturais:

- **Volume sem escala humana:** agendar manualmente milhares de tarefas, uma a uma, consome horas de advogados e estagiários que poderiam estar em atividades intelectuais mais qualificadas.
- **Heterogeneidade das fontes de entrada:** publicações chegam em planilhas de clientes (Onesid, Onerequest, spreadsheets) com formatos diferentes; nem todas cabem no Legal One sem tratamento.
- **Classificação e priorização manuais:** determinar se uma publicação é "Cumprimento de Sentença", "Audiência de Instrução" ou "Acórdão Não Provido" exige leitura humana — e humanos cansam, erram e divergem entre si.

O DunaFlow nasceu para fechar esse gap: ser a camada de orquestração que entende os formatos de entrada, aciona a inteligência artificial quando necessário, distribui o trabalho entre setores e squads, e conversa com o Legal One via API ou automação de interface quando não há API disponível.

### 2. O problema antes do DunaFlow

Antes da implantação do DunaFlow, a operação envolvia ao menos cinco pontos de atrito mensuráveis:

| Dor operacional            | Consequência                                         | Sintoma observado                                               |
|----------------------------|------------------------------------------------------|-----------------------------------------------------------------|
| Cadastro manual de tarefas | Tempo excessivo para cadastrar publicações em lote   | Advogados gastando horas em cadastros repetitivos               |
| Classificação manual       | Baixa consistência entre operadores                  | Mesma publicação rotulada de formas diferentes                  |
| Planilhas heterogêneas     | Retrabalho para normalizar cada batch                | Scripts ad-hoc em Excel para cada cliente                       |
| Falta de visibilidade      | Agendamentos perdidos e tarefas esquecidas           | Auditoria reativa, via reclamação do cliente                    |
| Limitações do Legal One    | Ações não disponíveis via API exigem operador humano | "Dar baixa" em publicações demandava sessão de operação na tela |

Cada uma dessas dores foi mapeada, instrumentada e, em diferentes graus, endereçada pela plataforma.

### 3. Proposta de valor do DunaFlow

A proposta de valor do DunaFlow se organiza em quatro grandes eixos:

#### 3.1. Automação de tarefas em larga escala

Agendamento de milhares de tarefas no Legal One a partir de múltiplas fontes (planilhas .xlsx, APIs Onesid e Onerequest), com validação prévia, tratamento de duplicidade e relatórios de sucesso/falha por item. O operador sobe a planilha, revisa o diagnóstico e confirma — o resto é com a plataforma.

#### 3.2. Inteligência artificial aplicada à triagem jurídica

Todas as publicações judiciais capturadas podem ser classificadas automaticamente usando modelos Claude da Anthropic. O pipeline aplica taxonomia jurídica consolidada da MDR (categorias como "1º Grau — Cível", "Audiência Agendada", "Tutela", "Sentença"), identifica polo, sub-categorias, nature de processo e sugere ações downstream. A combinação de IA com override manual garante qualidade sem perda de agilidade.

#### 3.3. Monitoramento proativo de processos

Integração com DataJud (endpoint oficial do CNJ) e DJEN/Comunica permite acompanhar movimentações e publicações além do recorte Legal One. Uma régua de maturidade em 7 níveis classifica cada processo (de MONITORED a CLOSURE\_CONFIRMED), e uma fila operacional sinaliza processos elegíveis para baixa — com rastreabilidade completa de evidências.

#### 3.4. Operação previsível e auditável

Toda execução deixa rastro: logs de batch, tentativas de classificação, eventos analíticos, falhas de RPA. Dashboards consolidam a visão do dia, permitindo que gestores detectem gargalos e fluxos travados antes de virarem problema.

### 4. Benefícios e impacto

| Dimensão                 | Antes                       | Depois do DunaFlow                | Como se mede                                |
|--------------------------|-----------------------------|-----------------------------------|---------------------------------------------|
| Tempo de cadastro        | Horas/dia                   | Minutos por lote                  | Logs de batch_execution                     |
| Erros de classificação   | Alto, variável por operador | Baixo e padronizado por taxonomia | Amostragem manual e taxa de reclassificação |
| Custo por publicação     | R\$/hora do operador        | ~ US\$ 0,002 via IA (Batch API)   | Relatório de tokens/consumo                 |
| Rastreabilidade          | Planilhas locais, e-mails   | DB PostgreSQL com auditoria       | Queries em publication_batch                |
| Tempo até detectar falha | Dias (cliente reclamando)   | Minutos (dashboard)               | Monitoramento interno                       |

Mesmo sem cifras fechadas, o efeito combinado é evidente: o DunaFlow libera horas de trabalho jurídico qualificado, padroniza a operação e substitui o "passo a passo no Legal One" por um processo industrializado — com inteligência no meio.

## Parte II — Visão Funcional (o que o sistema faz)

### 5. Visão geral da plataforma

A aplicação é estruturada em sete grandes módulos funcionais, todos acessíveis via interface web (React + TypeScript) e, quando aplicável, via API (FastAPI). Os módulos compartilham infraestrutura de autenticação, banco de dados e dashboards.

| Módulo                       | Para que serve                                                  | Páginas principais                                              |
|------------------------------|-----------------------------------------------------------------|-----------------------------------------------------------------|
| Orquestração de tarefas      | Criar tarefas em lote no Legal One a partir de planilhas e APIs | Criar por Planilha, Criar por Processo, Criar por Modelo, Lotes |
| Classificação de publicações | Aplicar IA para classificar publicações capturadas              | Publicações, Classificador                                      |
| Tratamento de publicações    | Automatizar a baixa/tratamento no Legal One via RPA             | Tratamento de Publicações                                       |
| Monitoramento processual     | Inteligência sobre processos com DataJud + DJEN                 | (Em evolução — tabelas analíticas prontas)                      |
| Automações agendadas         | Jobs recorrentes em background                                  | Automações                                                      |
| Administração                | Gestão de usuários, setores, squads, permissões                 | Admin, Perfil                                                   |
| Dashboards                   | Acompanhamento em tempo real                                    | Home, Dashboards por módulo                                     |

As próximas seções exploram cada módulo em detalhe — o problema que resolve, a jornada do usuário e os componentes técnicos envolvidos.

### 6. Módulo: Orquestração de Tarefas em Lote

#### 6.1. O que o módulo resolve

Advogados e gestores precisam criar centenas de tarefas no Legal One a partir de listas de processos, publicações ou solicitações externas. Fazer isso manualmente, um a um, é inviável em escala. O módulo de orquestração recebe essas listas em formatos diferentes e as transforma em tarefas Legal One devidamente vinculadas a processos, responsáveis, setores e datas.

#### 6.2. Estratégias de entrada

O módulo implementa o padrão Strategy. Cada fonte de entrada tem sua própria classe que sabe ler, normalizar e disparar a criação de tarefas. Hoje existem três estratégias em produção:

| Estratégia          | Fonte                               | Caso de uso                                                 |
|---------------------|-------------------------------------|-------------------------------------------------------------|
| SpreadsheetStrategy | Planilha .xlsx enviada pelo usuário | Operador recebe lista de um cliente e quer agendar em massa |
| OnesidStrategy      | API Onesid (parceiro)               | Integração recorrente com entrada programática de tarefas   |



|                    |                           |                                                      |
|--------------------|---------------------------|------------------------------------------------------|
| OnerequestStrategy | API Onerequest (parceiro) | Integração recorrente alternativa com lógica própria |
|--------------------|---------------------------|------------------------------------------------------|

Esse desenho facilita adicionar novas fontes — basta implementar uma nova classe que estenda `BaseStrategy`, sem tocar em nada mais.

### 6.3. Jornada do usuário (planilha)

1. Usuário acessa a página "Criar Tarefas por Planilha" e envia o arquivo .xlsx.
2. O sistema faz pré-análise: valida colunas, encontra linhas inválidas, detecta duplicidades contra o Legal One e exibe um relatório.
3. O usuário revisa, escolhe quais linhas confirmar e dispara o batch.
4. Um worker de background (`BatchExecutionWorker`) processa o lote de forma assíncrona, evitando travar o navegador.
5. A página de Lotes exibe o status em tempo real — totais processados, sucessos, falhas, motivo de cada falha.

### 6.4. Funcionalidades relacionadas

- Modelos de tarefa (Task Templates): grupos pré-configurados (ex.: "Pacote de cumprimento de sentença") que criam várias tarefas encadeadas de uma só vez.
- Criação por processo único (Lookup CNJ): entrada manual para uma única tarefa, com autocomplete do processo via Legal One.
- Regras e grupos de tarefas: definem rotinas recorrentes (`TaskRule`, `TaskGroup`) com controle de co-ocorrência.
- Sub-tipos de tarefa sincronizados com o Legal One, garantindo que os IDs que a API exige estejam sempre atualizados.

#### Padrão arquitetural

A orquestração segue o princípio "1 lote = 1 registro `BatchExecution`". Cada item dentro do lote gera um `BatchExecutionItem` com input de origem, output do Legal One, status e timestamps — o que dá ao operador uma trilha de auditoria completa e ao suporte uma visão clara para diagnosticar falhas.

## 7. Módulo: Classificação Inteligente de Publicações

### 7.1. O problema

Publicações judiciais chegam ao escritório como texto puro vindo dos diários oficiais. A classificação — "isto é uma intimação de sentença? é uma designação de audiência?" — é o que determina a ação a ser tomada. Feita manualmente, ela é lenta, inconsistente entre pessoas e quase impossível em grandes volumes.

## 7.2. A solução: pipeline com IA

O módulo de Classificação aplica um modelo de linguagem (Claude Haiku 4.5 da Anthropic) a cada publicação capturada, usando a taxonomia jurídica consolidada internamente. A taxonomia é extensa e reflete a prática real do escritório:

| Categoria principal                                   | Subcategorias (exemplos)                                                                        | Uso                                  |
|-------------------------------------------------------|-------------------------------------------------------------------------------------------------|--------------------------------------|
| 1º Grau — Cível / Execução                            | Apresentação de Contestação, Cumprimento de Sentença, Penhora, Alvará, Prescrição Intercorrente | Ações na fase de primeiro grau       |
| 2º Grau — Cível                                       | Contrarrazões, Agravo de Instrumento, Acórdão Provido / Não Provido, Decisão Monocrática        | Fase recursal                        |
| Tutela                                                | Tutela Concedida, Mantida, Revogada, Modificada, Não Concedida                                  | Medidas de urgência                  |
| Audiência Agendada                                    | Conciliação, Instrução, Não especificada                                                        | Designações de audiência com horário |
| Sentença                                              | Procedente, Improcedente, Parcialmente Procedente, Homologação                                  | Decisões de mérito                   |
| Manifestação das Partes, Provas, Custas, Para Análise | (conforme taxonomia)                                                                            | Demais atos processuais              |

## 7.3. Funcionalidades-chave

- Classificação automática em lote: dispara o classificador para todas as publicações ainda não classificadas.
- Classificação sob demanda: operador pode classificar uma publicação específica.
- Reclassificação manual inline: dropdown na tabela de publicações que altera categoria/subcategoria imediatamente, sem chamadas extras ao Legal One — fundamental para respeitar rate-limits.
- Preservação de override manual: a classificação primária recebe origem "manual\_override" quando vem de humano, impedindo que reprocessamentos automáticos sobrescrevam a decisão.
- Cache de consulta (skip\_responsible\_lookup): reclassificação não refaz lookups no Legal One, preservando dados já coletados e evitando 429.
- Edição de horário de audiência: modal de proposta de tarefa com campo de hora editável; o sistema recombina data + hora em ISO 8601 preservando cada parte isoladamente.
- Badge de UF/Tribunal: a jurisdição é derivada diretamente do número CNJ (J=8 → estadual; J=4 → federal; J=5 → trabalhista; J=6 → eleitoral), sem chamadas de API.

## 7.4. Resiliência e controle de custo

A integração com a API da Anthropic foi robustecida para dois cenários críticos:

- **Rate-limiting (429):** o cliente usa retry com backoff exponencial (15s → 30s → 60s, até 3 tentativas). Quando o header `retry-after` está presente, ele é respeitado. Isso elimina perdas silenciosas observadas em corridas iniciais.
- **Controle de concorrência:** `CONCURRENCY=1` e sleep de 12s entre chamadas para ficar confortavelmente abaixo do limite de 5 RPM da chave atual, com margem para expansão se a cota for elevada.

## 7.5. Custo observado

Um lote de referência com 145 publicações consumiu 533.822 tokens (514.669 input + 19.153 output). Via Batch API, o custo total ficou em torno de US\$ 0,25 — o equivalente a US\$ 0,0017 por publicação. Via Standard API, US\$ 0,37. Para 10 mil publicações anuais, o custo de IA é inferior a US\$ 40, tornando a classificação economicamente desprezível no orçamento de tecnologia.

### Decisão arquitetural: override manual imbatível

A plataforma assume que o humano sempre tem a palavra final. Se um operador reclassifica, essa decisão é marcada como `"manual_override"` e não pode ser sobrescrita por reprocessamentos automáticos. A IA complementa, não substitui.

## 8. Módulo: Tratamento de Publicações no Legal One (RPA)

### 8.1. Por que RPA

Algumas operações do Legal One — em particular, "dar tratamento" em publicações (marcar como tratada, como sem providência, anexar documento) — não têm endpoint REST público. A solução é automatizar a própria interface do CRM. O DunaFlow usa Playwright (navegação headless) para executar essas ações sem que um humano precise abrir a tela.

### 8.2. Fluxo de tratamento

6. Usuário (ou automação) enfileira publicações na `PublicationTreatmentQueue` com um alvo: `TRATADA` ou `SEM_PROVIDENCIAS`.
7. Uma execução (`PublicationTreatmentRun`) é criada com status `INICIANDO`.
8. O runner Playwright abre o Legal One, efetua login, navega até cada publicação e aplica a ação correspondente.
9. A cada item, o sistema registra sucesso ou falha com motivo, mantendo histórico completo.
10. A execução termina em `CONCLUIDO`, `CONCLUIDO_COM_FALHAS`, `FALHA` ou `INTERROMPIDO`.

### 8.3. Lições aprendidas e robustez

RPA é notoriamente frágil — qualquer mudança na UI do Legal One pode quebrar o fluxo. A evolução do módulo foi pautada por incidentes reais e resultou em cinco melhorias estruturais:

| Incidente observado                                        | Correção aplicada                                                                                              |
|------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| Upload caía no input errado quando a dropzone não aparecia | Fallback de input[type=file] em escopo de página agora é rejeitado; lança erro explícito                       |
| Confirmação do dropzone não era verificada                 | Agora existe verificação explícita do signal; modal fecha e erro é lançado se upload não confirmado            |
| Fluxo continuava com 0 anexos ao "salvar"                  | Verificação de linhas na grid de anexos — falha se < 2 linhas, em vez de seguir silenciosamente                |
| Loops infinitos de retry em falha de dropzone              | Erros de dropzone movidos para FINAL_NO_RETRY — não reentram na fila                                           |
| Timeouts excessivos no botão "Marcar como concluído"       | Timeout reduzido para 15s com cascata: força → dispatch_event. Chamada prévia de _settle() estabiliza a Ext.js |

O resultado é um runner muito mais previsível: falhas agora fazem falhar, em vez de prosseguir com estado inconsistente.

## 9. Módulo: Monitoramento de Inteligência Processual

### 9.1. Objetivo

Cobrir o que o Legal One não cobre — publicações e movimentos em fontes oficiais (DataJud e DJEN/Comunica) que permitem antecipar e auditar o ciclo de vida do processo. O módulo foi desenhado para entregar três coisas: carteiras monitoradas por cliente, régua de maturidade automatizada e fila operacional para validação de baixa.

### 9.2. Integrações

| Fonte           | O que o DunaFlow extrai                                                                                                                       |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| DataJud (CNJ)   | Snapshot do processo, classes, movimentos, tribunal, datas-chave. Suporta query Elasticsearch e cursor search_after para cargas incrementais. |
| DJEN / Comunica | Publicações, comunicações e certidões por caderno, comunicação ou certidão. Métodos genéricos para não depender de taxonomia externa.         |

### 9.3. Modelo de dados analítico

- MonitoringPortfolio — carteira monitorada por cliente / unidade / operação / segmento.
- MonitoredProcess — processo consolidado com status analítico, score, fila e timestamps de sincronização.
- ProcessRawPayload — camada bruta para auditoria, replay e reprocessamento.
- ProcessMovement — fato de movimentações (DataJud).

- ProcessPublication — fato de publicações, comunicações e certidões (DJEN/Comunica).
- ProcessAnalyticalEvent — regra disparada, impacto de score, justificativa, status de homologação.
- ProcessOperationalQueueItem — fila operacional derivada da classificação.
- IntegrationSyncRun e IntegrationSyncCursor — observabilidade e continuidade de cargas por fonte.

#### 9.4. Régua de maturidade (7 níveis)

| Nível | Status                           | Semântica                                             |
|-------|----------------------------------|-------------------------------------------------------|
| 0     | MONITORED                        | Processo ativo sendo acompanhado, sem sinal relevante |
| 1     | DECISION_EVENT                   | Publicação decisória detectada                        |
| 2     | RECURSAL_ATTENTION               | Sinal recursal em andamento — manter em atenção       |
| 3     | NEAR_TRANSIT                     | Indícios de proximidade de trânsito em julgado        |
| 4     | STRONG_TRANSIT_INDICATIVE        | Indicativo forte de trânsito em julgado               |
| 5     | ELIGIBLE_FOR_OPERATIONAL_CLOSURE | Elegível para baixa operacional                       |
| 6     | CLOSURE_CONFIRMED                | Baixa / arquivamento confirmado                       |

#### 9.5. Motor de score

O score é calculado combinando evidências extraídas das fontes. As regras iniciais são:

- Eventos decisórios aumentam score.
- Publicações e certidões agregam confiança.
- Trânsito em julgado expresso eleva fortemente a maturidade.
- Sinais recursais reduzem score e seguram o processo em fila de atenção.
- Janela sem impulso relevante aumenta a chance de encaminhamento à baixa.
- Confirmação de arquivamento definitivo/baixa definitiva fecha a régua em nível 6.

##### Por que esse desenho importa

A régua de maturidade traduz "aquilo que um coordenador jurídico olharia no processo" em um número. Isso permite priorizar filas, automatizar rotinas de cobrança/baixa e construir BI em cima de decisões homologadas — sem depender exclusivamente do que o Legal One sabe.

## 10. Módulo: Automações Agendadas

### 10.1. Motivação

Muitas rotinas precisam acontecer sem operador: puxar publicações novas de manhã cedo, classificar o que chegou, sincronizar metadados do Legal One a cada N horas. O módulo de Automações permite configurar esses jobs via interface, sem intervenção de DevOps.

### 10.2. Como funciona

Cada automação é um registro `ScheduledAutomation` com agendamento (cron ou intervalo em minutos) e uma lista de passos. O `APScheduler`, embarcado no backend FastAPI, registra e dispara os jobs. No startup da aplicação, todas as automações habilitadas são repovoadas — garantindo continuidade mesmo após reinícios.

### 10.3. Capacidades

- Agendamento cron: expressões completas (ex.: "0 7 \* \* 1-5" para dias úteis 7h).
- Agendamento por intervalo: "a cada 360 minutos" para rotinas não ancoradas em horário específico.
- Passos encadeados: "pull\_publications" → "classify" rodam na ordem definida.
- Escopo por escritório (office\_ids) e janela de lookback inicial ajustável por automação.
- Observabilidade: cada execução gera um `ScheduledAutomationRun` com status, duração e contagens.
- Recuperação de órfãos: no startup, execuções deixadas em "running" são marcadas como interrompidas, evitando zumbis no dashboard.

### 10.4. Exemplos reais

- "Diário 07:00 — puxar e classificar SP" — busca publicações desde o cursor, dispara classificação em lote.
- "A cada 6 horas — puxar publicações RJ" — carga incremental sem bloquear o pipeline do dia.
- "Sync metadados Legal One a cada 24h" — mantém listas de setores, usuários e escritórios sincronizadas.

## 11. Módulo: Administração, Usuários e Permissões

### 11.1. Estrutura organizacional

O DunaFlow modela a estrutura da MDR em três níveis navegáveis:

- Setores — agrupamentos macro (Contencioso Cível, Trabalhista, etc.).
- Squads — times dentro do setor com responsáveis definidos.
- Usuários — pessoas, com vínculos a squads e setores.

## 11.2. Autenticação e sessão

- Login por e-mail + senha hash (bcrypt).
- Tokens JWT para sessão; renovação e invalidação configuráveis.
- Rotas protegidas no backend (dependências FastAPI) e no frontend (ProtectedRoute).
- Trocar senha: dialog em Perfil, com validação no backend.

## 11.3. Sincronização com o Legal One

Usuários, escritórios e tipos de tarefa têm scripts dedicados de sincronização (`run_manual_sync.py`, `test_sync_users.py`, `test_sync_offices.py`, `test_sync_task_types.py`). Isso mantém o universo local alinhado com o CRM — evitando IDs órfãos em tempo de execução.

## 12. Módulo: Dashboards e Observabilidade

### 12.1. Dashboards por domínio

- Home: visão consolidada do dia (lotes, publicações classificadas, automações rodando).
- Lotes (`BatchExecutionsPage`): tabela com status, taxa de sucesso e drill-down por item.
- Publicações (`PublicationsPage`): listagem com filtros por status, categoria e jurisdição; ações inline (reclassificar, ver detalhes).
- Tratamento de publicações: fila e execuções com progresso em tempo real.
- Classificador: painel com backlog, taxa de classificação e custo acumulado.
- Automações: próximas execuções, histórico de runs, habilitar/desabilitar.
- Análise de planilhas: diagnóstico prévio ao envio de um batch.
- Monitor de fix de posições no Legal One: página específica para acompanhar correções multi-processo.

### 12.2. Observabilidade de baixa infra

O DunaFlow expõe métricas no formato Prometheus (via `prometheus.yml` empacotado). Isso permite que monitoring externo (Grafana, Datadog, etc.) capture saúde e performance sem que o sistema precise de middleware proprietário.

## Parte III — Visão Técnica

### 13. Arquitetura geral

O DunaFlow é uma aplicação web full-stack organizada em três camadas claramente separadas:

#### 13.1. Backend

Um serviço FastAPI (Python 3.10) que expõe uma API REST versionada (/api/v1). Ele concentra toda a lógica de negócio, acessa o banco PostgreSQL via SQLAlchemy e ORM, e orquestra integrações externas (Legal One, Anthropic, DataJud, DJEN). Workers em background (BatchExecutionWorker e APScheduler) rodam no mesmo processo para simplificar o deploy.

#### 13.2. Frontend

SPA construída com React 18 + TypeScript + Vite. Usa shadcn/ui sobre Tailwind CSS, react-router-dom para navegação e @tanstack/react-query para fetching e cache. O build estático é servido por Nginx, com proxy reverso para /api.

#### 13.3. Banco de dados

PostgreSQL gerenciado via Alembic para migrações versionadas. Hoje o projeto tem mais de 30 migrações aplicadas, cobrindo o schema inicial, tabelas de batch, classificação, cache de processos, tratamento de publicações, automações agendadas e monitoramento processual.

#### 13.4. Topologia de deploy

Produção roda em Docker Compose orquestrado pelo Coolify, com três serviços:

| Serviço  | Imagem / Dockerfile                | Função                        |
|----------|------------------------------------|-------------------------------|
| postgres | postgres oficial                   | Persistência                  |
| api      | Dockerfile (Python/FastAPI)        | Backend + workers + scheduler |
| frontend | Dockerfile.frontend (Vite → Nginx) | SPA estática e proxy /api     |

O override de docker-compose.override.yml é ignorado em produção e usado apenas em dev local (bind mounts + Vite dev server).

### 14. Stack tecnológico

| Camada  | Tecnologia  | Uso no DunaFlow                  |
|---------|-------------|----------------------------------|
| Backend | Python 3.10 | Linguagem da API e dos workers   |
| Backend | FastAPI     | Framework da API REST            |
| Backend | SQLAlchemy  | ORM + camada de acesso a dados   |
| Backend | Alembic     | Versionamento de schema          |
| Backend | Pydantic    | Validação de contratos e schemas |



|                 |                            |                                                        |
|-----------------|----------------------------|--------------------------------------------------------|
| Backend         | APScheduler                | Scheduler de automações                                |
| Backend         | Playwright                 | RPA para tratar publicações no Legal One               |
| Backend         | httpx                      | Client HTTP assíncrono (Legal One, Anthropic, DataJud) |
| Frontend        | React 18 + TypeScript      | UI reativa e tipada                                    |
| Frontend        | Vite                       | Build tool e HMR em dev                                |
| Frontend        | Tailwind CSS               | Estilização utilitária                                 |
| Frontend        | shadcn/ui + Radix          | Biblioteca de componentes acessíveis                   |
| Frontend        | @tanstack/react-query      | Fetching, cache e invalidation                         |
| Frontend        | React Router               | Roteamento SPA                                         |
| Frontend        | recharts                   | Gráficos nos dashboards                                |
| Dados           | PostgreSQL                 | Banco relacional principal                             |
| IA              | Anthropic Claude Haiku 4.5 | Classificação de publicações                           |
| Infra           | Docker / Docker Compose    | Runtime e composição de serviços                       |
| Infra           | Coolify                    | PaaS self-hosted para deploy                           |
| Infra           | Nginx                      | Proxy reverso e serve da SPA                           |
| Observabilidade | Prometheus                 | Métricas exportadas                                    |

## 15. Padrões de projeto

### 15.1. Strategy (orquestração)

BaseStrategy define o contrato (process\_batch) e cada fonte de entrada implementa a sua. Adicionar uma nova fonte é uma questão de criar nova classe + registrar na factory — sem tocar em código existente. Princípio aberto-fechado aplicado.

### 15.2. Repository implícito via SQLAlchemy

A camada de acesso a dados não cria um repositório formal, mas cada service encapsula suas queries. Isso mantém o domínio coeso (classificação não toca em batch, batch não toca em monitoramento) sem over-engineering.

### 15.3. Service layer

Serviços como BatchTaskCreationService, PublicationBatchClassifier, ScheduledAutomationService, PublicationTreatmentService concentram regras de negócio. Endpoints são finos: autenticam, validam e delegam.

## 15.4. Worker pool + scheduler

Long-running jobs (batches, RPA runs) não bloqueiam o request cycle. O BatchExecutionWorker faz polling em fila interna; o APScheduler dispara jobs agendados. Ambos convivem com o servidor ASGI.

## 15.5. Contratos de integração desacoplados

Para DataJud e DJEN, o módulo de process\_monitoring expõe contratos (DataJudProcessSnapshot, ComunicaPublicationRecord, DetectedEvidence, MonitoringEvaluation) que normalizam os payloads crus em estruturas homogêneas — o restante do sistema consome essas estruturas, não o formato externo.

## 16. Integrações externas

| Integração       | Protocolo                     | O que consumimos                                       | O que fazemos                                                      |
|------------------|-------------------------------|--------------------------------------------------------|--------------------------------------------------------------------|
| Legal One        | REST (API + OAuth) + UI (RPA) | Tarefas, processos, usuários, escritórios, publicações | Criamos tarefas, atualizamos publicações, disparamos ações via RPA |
| Anthropic Claude | REST + Batch API              | Modelo Haiku 4.5                                       | Classificamos publicações via prompts estruturados                 |
| DataJud (CNJ)    | Elasticsearch-like            | Snapshots de processos, movimentos                     | Populamos ProcessMovement e consolidamos MonitoredProcess          |
| DJEN / Comunica  | REST                          | Publicações, comunicações, certidões                   | Populamos ProcessPublication por caderno/comunicação/certidão      |
| SMTP (e-mail)    | SMTP                          | —                                                      | Notificações operacionais (via MailService)                        |

### 16.1. Por que duas estratégias para Legal One (API + RPA)

A API do Legal One cobre operações de cadastro e consulta, mas não todas as ações de tela — especialmente o "tratamento" de publicações, que tem fluxo específico de anexos e confirmações visuais. Como não abrimos mão da automação, o DunaFlow mantém dois canais: API sempre que possível (mais barato, mais estável) e RPA quando necessário (com engenharia defensiva extra).

### 16.2. DataJud e DJEN: comunicação robusta

O DataJud expõe busca Elasticsearch com paginação via search\_after — o client implementa cursor persistente (IntegrationSyncCursor) para cargas incrementais retomáveis. O DJEN/Comunica tem APIs de granularidade variada; o client oferece métodos genéricos por caderno, comunicação e certidão, com normalização posterior.

## 17. Modelo de dados

O schema PostgreSQL tem mais de 30 tabelas. Agrupadas por domínio, ficam claras as fronteiras do sistema:

### 17.1. Legal One e entidades canônicas

- legal\_one\_users, legal\_one\_offices, legal\_one\_task\_types — cache local da estrutura do CRM.
- canonical.\* — tabelas canônicas para entidades derivadas (processos, partes, indexações).
- associations.\* — relações many-to-many entre entidades.

### 17.2. Orquestração e lotes

- batch\_execution + batch\_execution\_items — cabeçalho e itens de cada lote (com fingerprint e controle de estado).
- task\_templates, task\_groups, task\_rules — modelos e regras de geração de tarefas.

### 17.3. Publicações e classificação

- publication\_batch, publication\_batch\_classification — lotes de publicações e resultados de classificação.
- publication\_capture, publication\_search — capturas e buscas na origem.
- publication\_record — registro consolidado (com campos como polo, UF, natureza\_processo, audiência, subcategoria secundária).
- classification\_\* — overrides, histórico e índices de performance.
- office\_lawsuit\_index, office\_classification — relações escritório/processo e classificações operacionais.

### 17.4. Tratamento de publicações no Legal One

- publicacao\_tratamento\_execucoes — cabeçalho de cada run RPA (status: INICIANDO, EXECUTANDO, PAUSADO, CONCLUIDO, CONCLUIDO\_COM\_FALHAS, FALHA, INTERROMPIDO).
- publicacao\_tratamento\_fila — fila por item com alvo (TRATADA, SEM\_PROVIDENCIAS) e status técnico.

### 17.5. Monitoramento processual

- monitoring\_portfolios, monitored\_processes, process\_raw\_payloads, process\_movements, process\_publications, process\_analytical\_events, process\_operational\_queue\_items, integration\_sync\_runs, integration\_sync\_cursors.

### 17.6. Agendamento automatizado

- scheduled\_automations + scheduled\_automation\_runs — definição dos jobs e histórico de execuções.

## 17.7. Cache de processos Legal One

- lawsuit\_cache — cache local para lookups frequentes, diminuindo pressão na API.

## 18. Segurança

- Autenticação por JWT assinado com segredo configurável em variável de ambiente.
- Senhas armazenadas com bcrypt (hash + salt) — nunca em texto claro.
- CORS middleware restrito a origins configurados no settings.
- Variáveis de ambiente (.env) fora do controle de versão; acesso a APIs externas via chaves por ambiente.
- Rotas administrativas (admin.py) com dependência extra de verificação de is\_admin.
- Logs estruturados para auditoria de ações críticas (criação de lotes, disparos de RPA, mudanças de permissão).
- Isolamento de workers: RPA roda com contexto Playwright em diretório temporário, com limpeza explícita após cada sessão.

### Boa prática aplicada

A plataforma distingue claramente status de negócio (publicação AGENDADA / IGNORADA) e status técnico (fila PENDENTE / PROCESSANDO / CONCLUIDO / FALHA). Isso é importante para auditoria: se um operador marca "ignorado" mas o RPA ainda tentou processar, o sistema consegue explicar as duas camadas de decisão independentemente.

## 19. Deploy e operação

### 19.1. Ambientes

- Dev local: docker-compose com override (bind mounts e Vite HMR).
- Produção: Coolify com docker-compose.yml em branch main.
- Variáveis: .env por ambiente, seguindo o modelo do .env.example.

### 19.2. Pipeline de deploy

11. Merge em main no GitHub.
12. Coolify detecta o push e reconstrói as imagens api e frontend.
13. Migrations Alembic são aplicadas antes do start da API.
14. Lifespan do FastAPI inicia worker, scheduler e repova automations habilitadas.
15. Nginx atualiza a SPA estática e segue proxyando /api.

### 19.3. Observabilidade

- Logs da aplicação em stdout — coletados pelo stack do Coolify.
- Prometheus scrape em /metrics (prometheus.yml empacotado).

- Dashboard interno mostra sinais de capture e falhas recorrentes.

#### **19.4. Rollback e manutenção**

- Alembic downgrade para reverter migrations (ex.: alembic downgrade pha004).
- Scripts utilitários: create\_user.py, run\_manual\_sync.py, update\_lawsuits.py, inspect\_lawsuit.py — cobrem operações rotineiras de backoffice.
- Compose volumes nominais permitem teardown controlado (docker volume rm) quando preciso.

## Parte IV — Evoluções, Decisões e Resiliência

### 20. Linha do tempo de evoluções relevantes

As entregas abaixo refletem marcos técnicos derivados de problemas reais — todos resolvidos e hoje em produção.

| Quando       | Tema          | Entrega                                                                 | Motivação                                          |
|--------------|---------------|-------------------------------------------------------------------------|----------------------------------------------------|
| Fase inicial | Orquestração  | Estratégias Onesid, Onerequest e Planilha consolidadas                  | Unificar três fluxos paralelos em uma única engine |
| Fase inicial | Admin         | Gestão de setores, squads e usuários                                    | Refletir a estrutura real do escritório no sistema |
| Evolução     | Classificação | Pipeline IA com Claude Haiku 4.5 e taxonomia MDR                        | Eliminar classificação manual                      |
| Evolução     | Classificação | Reclassificação inline sem rechamar Legal One                           | Preservar overrides e evitar 429                   |
| Evolução     | Publicações   | Badge de UF via CNJ (sem API call extra)                                | Gestão de carteira por jurisdição                  |
| Evolução     | Publicações   | Edição de horário de audiência no modal                                 | Preservar data/hora em edições parciais            |
| Evolução     | Automação     | APScheduler + repovoamento no startup                                   | Rotinas recorrentes sem operador                   |
| Evolução     | RPA           | Playwright runner para tratamento de publicações                        | Cobrir ação sem API no Legal One                   |
| Consolidação | RPA           | Cinco correções de robustez (upload, dropzone, anexos, retry, timeouts) | Eliminar loops infinitos e estados inconsistentes  |
| Consolidação | Permissões    | Limpeza de pacote RPA legado e unificação de UI                         | Remover débito técnico que bloqueava padronização  |
| Consolidação | Monitoramento | Modelo analítico + régua de maturidade                                  | Inteligência além do Legal One                     |
| Recente      | Operação      | Deploy em Coolify com compose prod/dev separados                        | Entregar produção sem depender de pipeline externo |

### 21. Decisões técnicas com racional

#### 21.1. Por que FastAPI

Async nativo, tipagem forte via Pydantic, geração automática de OpenAPI, curva de aprendizado suave e excelente integração com SQLAlchemy e httpx. Em uma aplicação com muitas integrações externas I/O-bound, o modelo async é uma vantagem direta.

## 21.2. Por que SQLAlchemy + Alembic em vez de ORM async

A combinação SQLAlchemy + Alembic ainda é o padrão-ouro para aplicações Python com modelagem complexa. O projeto usa o modo síncrono dentro de handlers async conforme necessário, com o benefício de ter uma ferramenta de migration madura e previsível.

## 21.3. Por que React + Vite + shadcn/ui

Vite entrega HMR instantâneo, shadcn/ui sobre Radix oferece componentes acessíveis e copiáveis (em vez de uma dependência opaca), e Tailwind cobre estilização consistente sem CSS-in-JS. Isso resulta em um frontend enxuto, fácil de evoluir, sem framework de design system proprietário.

## 21.4. Por que Claude Haiku 4.5 em vez de modelos menores / maiores

Haiku 4.5 entrega qualidade de classificação suficiente para a taxonomia MDR a um custo irrisório via Batch API. Modelos maiores (Sonnet, Opus) teriam precisão marginalmente superior a custo muito maior; modelos locais abertos teriam custo de infraestrutura e manutenção desproporcional ao volume.

## 21.5. Por que Batch API para classificação em massa

50% de desconto em input e output com SLAs de até 24h — compatível com a cadência diária do escritório. Para classificações interativas (por exemplo, reclassificar uma publicação na tela), o sistema usa a Standard API.

## 21.6. Por que manter RPA em Playwright mesmo com a fragilidade

A alternativa é esperar Legal One abrir endpoints — fora do nosso controle. Playwright oferece navegação headless confiável, suporte a esperas explícitas, captura de estado Ext.js e debugging por screenshots. Junto com as cinco camadas de robustez adicionadas, é uma solução custo-efetiva.

## 21.7. Por que Strategy para orquestração

Cada fonte de entrada (planilha, Onesid, Onerequest) tem particularidades de normalização. O Strategy isola essas particularidades e permite testes unitários focados. Uma nova fonte entra com uma nova classe, sem modificar as existentes.

## 21.8. Por que derivar UF do CNJ em vez de campo persistido

O número CNJ já carrega essa informação deterministicamente. Persistir seria replicação de dado, com risco de divergência. A derivação em runtime é barata (operação string) e sempre correta.

## 21.9. Por que separar status de negócio e status técnico nas filas RPA

São decisões diferentes: "o operador quer tratar" versus "o RPA conseguiu tratar". Uma publicação pode estar AGENDADA (negócio) mas FALHA (técnico) — e o sistema precisa saber explicar esse descompasso sem confundir os dois planos.

### 21.10. Por que APScheduler dentro do mesmo processo

Um cron externo funcionaria, mas exigiria mais infraestrutura. APScheduler embarcado aproveita o mesmo banco para estado (via lifespan) e simplifica o deploy. Em escala maior, a migração para uma solução dedicada (Celery Beat, Temporal) é trivial — os jobs são funções puras que podem ser chamadas por qualquer scheduler.

## 22. Resiliência e tratamento de incidentes

A evolução do sistema foi, em boa parte, um aprendizado por incidentes — e cada aprendizado virou engenharia. Alguns dos padrões já absorvidos:

### 22.1. Rate-limit e backoff

Classificação com Claude: backoff exponencial (15 → 30 → 60s), respeito a retry-after, até 3 tentativas. Lookups Legal One com 429: a reclassificação manual foi redesenhada para não chamar o Legal One, eliminando a fonte principal do problema.

### 22.2. Estado explícito em RPA

Em vez de continuar "por inércia" em falhas silenciosas, cada passo do RPA verifica explicitamente o estado (upload confirmado, número de anexos na grid, resposta do botão "concluir"). Qualquer inconsistência interrompe a execução com erro claro.

### 22.3. Listas de erros retryables x finais

Classificação dos erros em duas listas: os que valem retry automaticamente e os que não. Erros de dropzone foram movidos para FINAL\_NO\_RETRY depois que causaram loops infinitos — um fix pequeno com impacto grande.

### 22.4. Recuperação de órfãos no startup

Se um run termina em "running" por crash do processo, ele é marcado como INTERROMPIDO no próximo startup. Dashboards não ficam mentindo; operadores sabem exatamente o que aconteceu.

### 22.5. Logs e audit trail

Tabelas de batch, execuções e runs funcionam como log estruturado: quem disparou, quando, quantos itens, o que falhou. Em incidentes, a primeira pergunta ("o que aconteceu ontem às 14h37?") é respondida em segundos.



## Parte V — Resultados e Roadmap

### 23. Resultados mensuráveis

#### 23.1. Custo de classificação

Métrica colhida em corrida real:

| Indicador                         | Valor               | Observação                          |
|-----------------------------------|---------------------|-------------------------------------|
| Publicações classificadas no lote | 145                 | Corrida de validação                |
| Tokens consumidos (total)         | 533.822             | 514.669 input + 19.153 output       |
| Custo via Batch API               | ~ US\$ 0,25         | 50% de desconto                     |
| Custo via Standard API            | ~ US\$ 0,37         | Referência para operação interativa |
| Custo por publicação              | US\$ 0,0017 – 0,003 | Dependendo da API                   |
| Projeção: 10.000 pub./ano         | ~ US\$ 20 – 30      | Ordem de grandeza                   |

#### 23.2. Eliminação de débito técnico

- Remoção completa do pacote RPA legado que bloqueava a unificação da UI.
- Limpeza da migração pha005 (rollback) e purga de configurações órfãs em core/config.py.
- Docker volumes órfãos documentados para teardown controlado.
- Resultado: frontend com Publicações e Tarefas em paridade funcional, sem duas UIs concorrentes.

#### 23.3. Robustez operacional

- Zero ocorrências recorrentes de loops infinitos de retry após a classificação dos erros em FINAL\_NO\_RETRY.
- Tempo de execução do "marcar como concluído" reduzido de 60s (default Playwright) para 15s com cascata.
- Publicações submetidas com 0 anexos — eliminado: sistema agora falha em vez de prosseguir.

#### 23.4. Cobertura funcional

A plataforma hoje entrega sete módulos em produção, três deles com camadas de IA/automação avançada (classificação, RPA, monitoramento analítico). O número de automações agendadas e a volumetria de batches são métricas vivas nos dashboards internos.

## 24. Roadmap — próximos passos sugeridos

### 24.1. Curto prazo (próximas semanas)

- Expor endpoints FastAPI para carteiras de monitoramento, resumo executivo, fila operacional e detalhe de processo (arquitetura de dados já pronta).
- Criar migrations Alembic para persistir snapshots, eventos e itens de fila do módulo de monitoramento processual.
- Jobs periódicos de sincronização DataJud e DJEN/Comunica com cursor persistente.
- Página frontend dedicada ao monitoramento processual (portfólios, drill-down, fila de baixa).

### 24.2. Médio prazo

- Views/marts de BI derivados das tabelas analíticas (Power BI / Metabase).
- Homologação de eventos analíticos por usuário — transformar "sugestão de baixa" em "baixa confirmada" com audit trail.
- Ampliar a taxonomia de classificação com novas categorias observadas em produção (feedback loop com operadores).
- Integração oficial com canais de notificação (Microsoft Teams / e-mail) para alertas de falhas de automação.

### 24.3. Longo prazo

- Multi-tenant / multi-escritório — preparar o schema para rodar mais de uma estrutura organizacional.
- Modelo próprio de reclassificação fine-tuned sobre histórico MDR, reduzindo ainda mais o custo por publicação.
- Exposição de insights (BI em tempo real) direto dentro da plataforma, sem dependência de ferramenta externa.
- Expansão do RPA para outras operações Legal One sem API (juntar documentos, assinar, finalizar fases).

#### Visão de longo prazo

O DunaFlow tende a se consolidar como a camada "cérebro" do escritório — enquanto o Legal One é o sistema de registro, o DunaFlow é onde a inteligência, a orquestração e a observabilidade acontecem. Quanto mais rotinas forem capturadas nessa camada, mais previsível, auditável e escalável a operação se torna.

## Parte VI — Histórico de Atualizações

### 25. Visão geral do changelog

Este capítulo consolida o histórico de mudanças relevantes aplicadas ao DunaFlow após a publicação da versão 1.0 deste documento. A ideia é manter uma trilha clara entre o que está escrito nas partes anteriores e o que já evoluiu: cada versão descreve, por tema, as funcionalidades novas, as correções estruturais e as melhorias de performance ou UX. Mudanças menores (micro-ajustes de layout, pequenos bugs isolados) ficam fora do changelog para preservar a leitura executiva; mudanças estruturais e funcionais são sempre incluídas.

#### Como ler este capítulo

Cada versão recebe sua própria seção. Dentro de cada versão, os blocos são organizados por tema (Classificador, Publicações, Performance, Admin, etc.) para que leitores executivos consigam pular direto para a área de interesse. Sempre que aplicável, indicamos o impacto do lado do negócio e a contraparte técnica.

### 26. Versão 1.1 — Abril/2026

Primeira atualização após a v1.0. O ciclo foi dominado por cinco preocupações claras: preparar a aplicação para uso por equipe (concorrência, pool de conexões, índices), criar um loop de feedback contínuo para o classificador, tratar publicações obsoletas automaticamente, blindar integrações com o Legal One contra 429 e robustecer a camada de RPA. Abaixo estão as entregas consolidadas por tema.

#### 26.1. Classificador — sistema de feedback e auto-reparo

A evolução mais significativa do classificador nesta versão é o fechamento do ciclo "IA → operador → IA". Antes, uma reclassificação manual corrigia apenas o registro em questão; agora, ela alimenta explicitamente o modelo.

- **Feedback implícito:** toda vez que o operador reclassifica manualmente, a correção é registrada como exemplo para o classificador.
- **Feedback explícito:** novo botão "thumbs-down" com modal de correção onde o operador aponta categoria correta, polo, natureza e justificativa/regra.
- **Few-shot injection:** os feedbacks são injetados como exemplos no prompt (tanto na busca individual quanto no batch), melhorando a assertividade ao longo do tempo sem necessidade de re-treino.
- **Auto-reparo de classificações:** novo procedimento `repair_classification` corrige subcategorias na categoria errada (move para a categoria correta) e aplica fallback genérico quando o modelo devolve "-" ou "Para Análise" em categorias com subcategorias obrigatórias.

- **max\_tokens elevado:** saída do modelo ampliada de 1024 para 2048 tokens, eliminando truncamento de JSON em lotes mais complexos.

### Impacto

Pela primeira vez o classificador "aprende" com a operação diária. Cada correção entra no prompt e passa a influenciar decisões futuras — o custo de manutenção da qualidade cai e a acuracidade tende a subir com o volume.

## 26.2. Publicações — busca, filtros, agrupamento

- **Descarte automático de publicações obsoletas:** publicações com data anterior à criação da pasta do processo no Legal One são marcadas como DESCARTADO\_OBSOLETA e enfileiradas para o RPA como "sem providência". Elimina poluição na esteira processual.
- **Natureza processual para publicações sem pasta:** o classificador agora detecta a natureza em registros que ainda não foram vinculados a processos, permitindo triagem antes do cadastro da pasta.
- **Filtro "apenas sem processo":** tanto na busca quanto na classificação em lote, focando operadores no backlog de vinculação.
- **Filtro UF server-side e contagem total:** o filtro por unidade federativa agora roda no banco (não em memória) e a tabela exibe o total verdadeiro de registros.
- **Agrupamento individual para publicações sem processo:** em vez de agrupar por categoria + data, cada publicação sem processo vira uma linha. Preserva o contexto que seria perdido no agrupamento.
- **Bulk confirm / ignore em Processos com Publicações:** novas ações em massa para confirmar ou ignorar publicações de uma pasta.
- **Barra de progresso em tempo real durante a busca:** feedback visual durante buscas longas, com atualização granular do andamento.
- **total\_found corrigido:** antes, re-execuções da mesma janela mostravam o total cru da API (ex.: 732/0 duas vezes seguidas). Agora  $total\_found = new\_count + discarded\_count$ , refletindo apenas os registros efetivamente criados pela busca.
- **Export Excel polido:** altura de linhas fixada em 25 pontos e eliminação das "linhas em branco infinitas" (zeroHeight) que antes estragavam a apresentação.

## 26.3. Performance e escalabilidade (preparação para equipe)

Conjunto coordenado de mudanças para suportar 10+ usuários simultâneos sem degradação:

| Mudança                                                  | Motivo / efeito                                                                  |
|----------------------------------------------------------|----------------------------------------------------------------------------------|
| SQLAlchemy: pool_size=15, max_overflow=10, pool_pre_ping | Conexões reutilizadas com saúde verificada; evita "stale connection" após pausas |
| Uvicorn com múltiplos workers                            | Paraleliza requests entre processos, aproveitando CPUs                           |

|                                                                                    |                                                                                                                                |
|------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <b>(UVICORN_WORKERS=4)</b>                                                         |                                                                                                                                |
| <b>Postgres: max_connections=200, shared_buffers=512MB e tunings</b>               | Suporta a carga simultânea prevista sem contenção                                                                              |
| <b>Migration perf001 — 7 índices (simples e compostos) em publicacao_registros</b> | Queries de listagem e filtro passam a ser lineares no resultado, não no total                                                  |
| <b>Coluna "uf" materializada em publicacao_registros (perf002)</b>                 | Filtro UF vai direto ao WHERE; data migration popula registros existentes via SQL                                              |
| <b>Paginação de grupos no banco (list_records_grouped)</b>                         | Passa a buscar group_keys da página e só depois carrega records daqueles grupos — fim do "load all, group in Python"           |
| <b>62 endpoints convertidos de async def → def</b>                                 | Endpoints com I/O síncrono ao banco agora rodam em threadpool, liberando o event loop para requests concorrentes               |
| <b>Apply de batch com sessão própria em background</b>                             | Corrige "Instance is not persistent within this Session" — o background task agora abre SessionLocal() e recarrega o batch ali |

### Impacto

A plataforma passa de "aplicação para um operador" para "plataforma pronta para equipe". Essas mudanças são pré-requisito para o roll-out mais amplo e para a adoção transversal entre setores do escritório.

## 26.4. Resiliência — Legal One, Chromium e 429

- **Rate limiter global + sync sequencial:** nova camada centralizada protege todas as chamadas ao Legal One contra 429, inclusive durante sincronizações. Elimina o padrão anterior em que diferentes serviços disputavam a mesma janela de RPM.
- **shm\_size=1GB para o Chromium:** evita crashes do browser durante sessões RPA mais longas. Sem aumentar a memória compartilhada, o Chromium podia morrer silenciosamente sob carga.
- **Docker — cache de layers do Playwright/Chromium:** build otimizado para reaproveitar layers pesadas do Chromium, reduzindo tempo de rebuild.
- **Fallback por e-mail no sync de usuários:** quando o sync encontra um usuário criado manualmente sem ID Legal One, faz match por e-mail em vez de estourar UniqueViolation.

## 26.5. Admin — pré-aquecimento de caches

Um novo painel de operação em Admin resolve um dos gargalos recorrentes: a primeira busca do dia sempre era lenta porque o lawsuit\_cache estava frio.

- **Dashboard de cache:** barras de progresso, polling em tempo real e status por escritório, visibilidade completa do estado do cache.

- **Botão de pre-warm:** endpoint POST /admin/sync-caches que, em background, carrega o índice de processos por escritório e popula o lawsuit\_cache com CNJ e creationDate — economizando chamadas à API nas buscas de publicações subsequentes.

## 26.6. Lookup por CNJ — auditoria rica

- **Dados do processo no topo:** data de criação da pasta, escritório e metadados Legal One são exibidos em linha com a busca.
- **Timeline cronológica:** eventos do processo em ordem temporal, em um único eixo visual.
- **Cards expansíveis por publicação:** classificação IA, justificativa e status RPA em um drill-down que antes exigia navegação por várias telas.

## 26.7. UX — dialog de agendamento

- **Texto completo da publicação no topo do dialog:** evita que o operador precise voltar à lista para conferir o conteúdo antes de agendar.
- **Ícone inline de ignorar removido:** agora só o botão do grupo realiza a ação — reduz cliques acidentais.
- **Agendar a partir de IGNORADO:** rede de segurança — se algo foi ignorado por engano, ainda dá para agendar sem reclassificar.
- **due\_date\_reference nos templates:** novo campo com valores "publication" (prazo a partir da data da publicação, comportamento antigo) ou "today" (prazo a partir de hoje). Migration tpl003 adiciona a coluna com default retrocompatível.
- **Overflow no modal:** max-h com overflow-y-auto impede que o dialog extrapole a janela do navegador em telas pequenas.
- **SelectItem Radix com value vazio corrigido:** elimina erros do Radix UI ao renderizar selects opcionais.

## 26.8. Infraestrutura e scripts operacionais

- **TwoTask Apex — preparação de deploy:** ajustes para publicação em ambiente Apex/produção consolidada (ainda sob o nome legado TwoTask; o rebrand para DunaFlow ocorreu na v1.4).
- **create\_superuser.py:** novo script para criação do primeiro administrador em setups iniciais, complementando o create\_user.py existente.
- **Correções de migration (UF/CASE):** ajustes no CASE da migration perf002 para respeitar a regra do PostgreSQL que não permite referenciar aliases da mesma subquery — adicionado nível intermediário de subquery para materializar os aliases.

## 26.9. Resumo dos commits desta versão

A tabela abaixo consolida os 25 commits desta versão para referência rápida:

| Hash | Tema | Descrição curta |
|------|------|-----------------|
|------|------|-----------------|

|         |               |                                                                     |
|---------|---------------|---------------------------------------------------------------------|
| 02d9d42 | Classificador | Auto-reparo de classificações + grupos individuais sem processo     |
| 64caf61 | Publicações   | Publicações sem processo como grupos individuais + fix Excel        |
| 7836dfb | Classificador | Sistema de feedback + fix export Excel                              |
| fd4e215 | Classificador | Sistema de feedback para melhoria contínua                          |
| 238009d | Busca         | Barra de progresso em tempo real                                    |
| 9d33e91 | Publicações   | Filtro "apenas sem processo"                                        |
| f6ea7df | UX            | Dialog de agendamento: texto, ignora→agenda, prazo a partir de hoje |
| 7e32859 | Publicações   | Natureza processual em publicações sem pasta + filtro de vínculo    |
| bfd7f2b | Resiliência   | Rate limiter global + sync sequencial (Legal One)                   |
| f528171 | Admin         | Dashboard de cache com barras de progresso                          |
| 856d4c3 | Admin         | Pre-warm de caches (escritórios + creationDate)                     |
| 3ee3e39 | Fix/UX        | SQL alias na migration UF + overflow no modal                       |
| d8688a3 | Migration     | Correção de alias no CASE (Postgres)                                |
| 500ddbd | Lookup        | Auditoria rica com timeline e dados do Legal One                    |
| 43fa6a7 | Publicações   | Descarte automático de publicações obsoletas                        |
| 0e94d64 | Performance   | Coluna UF materializada + paginação de grupos no banco              |
| a506c35 | Sync          | Fallback por e-mail evita UniqueViolation                           |
| 7c46149 | Scripts       | create_superuser.py para setup inicial                              |
| 454aa61 | Publicações   | total_found conta registros vinculados                              |
| f1d3c35 | Performance   | async def → def em endpoints sem await                              |
| 3e3ebeb | Performance   | Pool de conexões, múltiplos workers e índices                       |
| ec1d6fe | Publicações   | Apply de batch com sessão própria                                   |
| a5ec571 | Docker        | Cache de layers Playwright/Chromium                                 |
| 8e49d68 | Publicações   | Filtro UF server-side e contagem total                              |

|         |             |                                                  |
|---------|-------------|--------------------------------------------------|
| c2135bb | Publicações | Bulk confirm/ignore em Processos com Publicações |
| 85e07f0 | Fix UI      | SelectItem com value vazio (Radix UI)            |
| 279e35a | Infra       | shm_size=1GB evita crash do Chromium             |
| 9637164 | Deploy      | Prepare TwoTask Apex deploy (pré-rebrand)        |

## 27. Como este documento é mantido

A proposta é que o documento evolua em versões curtas, alinhadas a ciclos de entrega. A cada ciclo (semanal, quinzenal ou mensal, conforme ritmo da equipe), uma nova subseção dentro deste capítulo consolida as entregas. O método recomendado:

16. Ao encerrar um ciclo, listar os commits relevantes desde o último corte registrado no documento.
17. Agrupar por tema (Classificador, Publicações, Performance, Admin, UX, Resiliência, Infraestrutura, etc.).
18. Para cada tema, escrever um bloco curto com os pontos em formato de lista, sempre começando pelo "o que mudou" e, quando útil, adicionar um callout de "impacto" com a leitura executiva.
19. Incluir a tabela-resumo de commits ao final da versão, servindo como índice rastreável até o git.
20. Incrementar a versão na capa (p.ex. 1.1 → 1.2) e ajustar a data.
21. Manter tudo em português técnico-executivo — o documento é, ao mesmo tempo, pitch interno e referência viva.

### Princípio

O que entra neste capítulo é o que altera a forma como a plataforma é usada, operada ou escalada. Correções pontuais sem impacto estrutural podem ficar só no git; o documento concentra mudanças que importam para a narrativa do produto.

## 28. Versão 1.2 — Abril/2026

Micro-ciclo dedicado à ergonomia da tela de Publicações. O foco foi diminuir o esforço visual do operador na triagem diária: filtros mais rápidos de achar e usar, uma nova dimensão de filtragem (natureza do processo) e a visibilidade do CNJ mesmo quando a publicação ainda não está vinculada a uma pasta.



## 28.1. Publicações — filtros e UX

- **Barra de filtros reorganizada em duas linhas, com labels explícitas:** cada controle passa a ter um rótulo visível acima, reduzindo o tempo que o operador gasta para entender o que cada filtro faz. A densidade cai, a leitura sobe.
- **Botão "Limpar filtros":** resetar o estado da tela em um clique. Elimina o padrão anterior de "recarregar a página quando perdi onde estava".
- **Novo filtro dinâmico por natureza do processo:** combo populado automaticamente com as naturezas detectadas no backlog (Embargos, Agravo, Cumprimento de Sentença, etc.). Permite cortes por tipo de ação judicial sem depender de tags manuais.
- **CNJ em publicações sem processo vinculado:** antes, registros sem pasta no Legal One escondiam o número. Agora o CNJ é exibido em linha — o operador consegue identificar e atuar sobre o registro mesmo antes da vinculação.
- **Thumbs-down movido para a coluna Status:** o controle de feedback explícito do classificador (introduzido na v1.1) saiu da linha principal e foi incorporado à coluna de status. Menos poluição visual no grid, sem perder o acesso rápido.

### Impacto

O módulo de Publicações já era o mais denso visualmente da plataforma — com classificação, polo, data, categoria, subcategoria, status, ações. A v1.2 é uma correção de rota ergonômica: o mesmo grid, com as mesmas informações, mas com menos atrito cognitivo por registro triado. Em operação sustentada (centenas de publicações por dia), o ganho composto é significativo.

## 28.2. Resumo dos commits desta versão

| Hash    | Tema             | Descrição curta                                                                                                   |
|---------|------------------|-------------------------------------------------------------------------------------------------------------------|
| 5723a69 | Publicações / UX | Filtro natureza, CNJ em publicações sem processo, reorganização da barra de filtros, thumbs-down na coluna Status |

## 29. Versão 1.3 — Abril/2026

Ciclo focado em amadurecer o módulo de Publicações e blindar o pipeline contra modos de falha discretos. A triagem diária ganhou uma nova dimensão de filtragem (polo: ativo/passivo), atalho direto para a pasta no Legal One a partir do modal de detalhe, retentativa por item na fila de tratamento, extração de CNJ a partir do próprio texto em publicações ainda não vinculadas e recuperação de classificações vindas com JSON truncado por max\_tokens. A base administrativa também melhorou: o índice de processos do admin passou a exibir o caminho hierárquico completo do escritório, em vez de apenas a folha.

### 29.1. Publicações — polo, link L1 e tarefa avulsa

- **Filtro dinâmico por polo:** combo adicional, ao lado do filtro por natureza, permite segmentar ativo/passivo sem sair da tela. Os valores são populados a partir do próprio backlog, seguindo o mesmo padrão do filtro de natureza introduzido na v1.2.
- **Link direto para o Legal One no modal de detalhe:** quando a publicação tem pasta vinculada, o modal exibe um atalho que abre o processo no Legal One em outra aba — útil para validar detalhes antes de confirmar o agendamento.
- **Criação de tarefa avulsa a partir da publicação:** além dos fluxos por template, agora é possível criar uma tarefa ad-hoc diretamente do modal da publicação, cobrindo casos em que nenhum template existente serve.
- **Padronização dos headers das páginas:** cabeçalhos de Publicações, Tratamento e Classificador passam a seguir a mesma estrutura visual — título, subtítulo, conjunto de ações à direita. Reduz a carga cognitiva ao navegar entre os módulos.

### 29.2. Publicações sem processo — extração de CNJ do texto

Para registros que chegam do DJEN/Comunica sem pasta vinculada no Legal One, o backend agora roda um extrator de CNJ em cima do próprio texto da publicação. O CNJ detectado é persistido no registro e usado para (i) exibir o número no grid e no modal, (ii) permitir que o operador abra o processo em ferramentas externas e (iii) habilitar vinculação automática quando o processo for cadastrado. A extração usa a expressão regular oficial do padrão CNJ e tolera variações tipográficas comuns (hífens, espaços).

#### Por que importa

A v1.2 fez o CNJ aparecer no grid quando a publicação já trazia o número. A v1.3 cobre o caso anterior — quando o número só está presente no corpo da publicação. Juntas, as duas versões eliminam o ponto cego de "publicação chegou, mas não sei de que processo é".

### 29.3. Tratamento de Publicações — retry individual

- **Retry por item falhado:** a fila de tratamento passa a suportar retentativa individual. O operador escolhe um item específico em FALHA e dispara um novo attempt sem precisar reenqueuear o lote inteiro.
- **Título padronizado da tela:** ajuste cosmético que alinha o header de Tratamento com os demais módulos (parte da padronização de headers descrita acima).

### 29.4. Classificador — robustez a arrays truncados

Em lotes grandes, o modelo pode atingir o limite de max\_tokens e devolver uma resposta com o array JSON truncado no meio. Antes, isso causava falha de parse e o lote inteiro ficava sem classificação. Agora, o classificador:

- Detecta o truncamento pelo stop\_reason = "max\_tokens".

- Tenta reparar o JSON fechando colchetes e chaves pendentes.
- Recupera as classificações parciais já emitidas antes do corte.
- Mantém as publicações restantes como "pendentes" para novo ciclo, em vez de perder o lote inteiro.

## 29.5. Admin e sincronizações

- **Path completo de escritório no índice de processos:** o admin passa a exibir a hierarquia completa (ex.: "MDR Advocacia / Contencioso Cível / SP — Bancário") em vez de apenas a folha ("SP — Bancário"). Elimina ambiguidade entre escritórios com folhas de nome parecido.
- **Reaproveitamento de syncs órfãs:** no sync de escritórios, registros deixados em estado intermediário por crashes anteriores são reaproveitados em vez de gerar duplicatas ou UniqueViolation.

## 29.6. Resumo dos commits desta versão

| Hash    | Tema                  | Descrição curta                                                              |
|---------|-----------------------|------------------------------------------------------------------------------|
| 08ec852 | Admin                 | Path completo do escritório no índice de processos                           |
| 9271767 | Classificador         | Recupera classificações de arrays truncados por max_tokens                   |
| 7982ec8 | Publicações           | Extraí CNJ do texto em publicações sem processo vinculado                    |
| 57dd048 | Tratamento            | Retry individual de falhas + padronização de título                          |
| e039201 | Publicações           | Filtro por polo + link L1 no modal                                           |
| 4ba58a3 | Publicações           | Filtro polo, link L1, padronização de headers                                |
| f45efce | Publicações / Offices | Filtro polo + link L1 + tarefa avulsa; reap sync órfãs; headers padronizados |

## 30. Versão 1.4 — Abril/2026

Versão de rebrand. A plataforma antes conhecida como TwoTask passa a se chamar DunaFlow, parte da identidade DUNATECH — o domínio tecnológico que concentra as iniciativas de produto da MDR. Além do rebrand, o ciclo entregou a reconstrução da Home como dashboard executivo, refinamentos de branding (favicon, sidebar, wordmark) e ajustes de responsividade para uso em celular. Nenhuma funcionalidade de negócio foi removida — tudo o que estava na v1.3 continua, agora sob a nova identidade visual.

### Decisão estratégica

"TwoTask" descrevia o ponto de partida — criar tarefas em lote no Legal One. A plataforma, hoje,

orquestra publicações, classificação, monitoramento processual e automações agendadas. O nome novo — DunaFlow — reflete esse movimento contínuo de dados e decisões pelo sistema, e por isso o rebrand foi escolhido como marco da v1.4.

### 30.1. Rebrand para DunaFlow by DUNATECH

- **Paleta DUNATECH:** navy (#0A1940) como cor dominante, blue elétrico (#1E7BFF) como accent interativo e blue profundo (#1558CC) como camada intermediária. A paleta substitui o azul genérico da v1.0-v1.3 por uma identidade mais fria, executiva e reconhecível.
- **Wordmark "Duna" + "Flow":** a primeira metade do nome fica em peso regular/bold e a segunda ganha tratamento itálico em blue elétrico — comunicando tanto solidez (base "Duna") quanto movimento (variação "Flow").
- **Wordmark "Flow" com efeito neon pulsante:** no header da aplicação, a parte "Flow" recebe um efeito de neon pulsante sutil, reforçando a ideia de fluxo vivo sem virar ruído visual. A fonte escolhida é Jura, para dar acabamento técnico.
- **Favicon DunaFlow:** novo favicon quadrado derivado da "onda" da marca — representa a identidade em formatos de 16×16 e 32×32 sem perder reconhecibilidade.
- **Sidebar com logo DUNATECH + wordmark DunaFlow:** no sidebar, o logo DUNATECH fica empilhado acima do wordmark DunaFlow — deixa explícito o vínculo da plataforma com o domínio DUNATECH, sem ofuscar a identidade do produto.

### 30.2. Dashboard executivo — Home reconstruída

A Home foi redesenhada para deixar de ser apenas um "menu inicial" e passar a ser uma tela de leitura executiva. O objetivo é que um coordenador ou sócio, ao abrir a plataforma, entenda em menos de 30 segundos onde estão os gargalos.

| Bloco da Home               | O que mostra                                                                                                                                                    |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| KPIs do topo                | Publicações do dia, publicações tratadas, backlog classificado, taxa de sucesso de RPA, tempo médio do ciclo                                                    |
| Funil de publicações        | Capturadas → classificadas → triadas → tratadas, com contagens e percentuais de conversão em cada etapa                                                         |
| Velocidade e throughput     | Curva de publicações/hora e tarefas/hora em janelas configuráveis, destacando horário de pico e vales                                                           |
| Saúde das capturas (rodapé) | CaptureHealthWidget: últimos ticks do poller por origem, falhas recentes, cursor atual. Foi movido para o rodapé para não competir com o foco executivo do topo |

#### Impacto

A home passa a ser um "painel de comando" em vez de um "menu". O custo marginal é zero — os

dados já eram coletados pelos demais módulos; a v1.4 só os consolidou em uma tela.

### 30.3. Templates — overrides de classificação no grid de cobertura

O grid de cobertura de templates, que mostra quais templates estão associados a cada categoria/subcategoria, agora respeita os overrides manuais de classificação. Antes, se o operador reclassificasse uma publicação, o grid ainda contabilizava a classificação original da IA — dando uma visão distorcida da cobertura. Agora, o grid é a fonte de verdade pós-override.

### 30.4. Responsividade em Admin e Publicações

- Admin com layout adaptativo em telas <768px — cards e tabelas colapsam em stack vertical, sem perder informação.
- Publicações em viewport estreito — colunas opcionais são escondidas, ações primárias ficam no topo e o modal ocupa a tela quase inteira em mobile.
- Nenhuma funcionalidade é restrita a desktop — a ideia é que um coordenador em trânsito consiga triar publicações pelo celular sem perda de autonomia.

### 30.5. Resumo dos commits desta versão

| Hash    | Tema      | Descrição curta                                         |
|---------|-----------|---------------------------------------------------------|
| 653695f | Branding  | Rebrand para DunaFlow com identidade DUNATECH           |
| be1ea99 | Branding  | Logo DunaFlow sem wordmark no sidebar                   |
| 44c94b2 | Branding  | Empilha logo DUNATECH + wordmark DunaFlow no sidebar    |
| 85e2720 | Branding  | Favicon DunaFlow quadrado a partir da onda da marca     |
| c35f912 | Dashboard | Home com KPIs, funil e velocidade + ajustes de branding |
| 6c4361e | Branding  | Wordmark 'Flow' com efeito neon pulsante                |
| e95a671 | Dashboard | CaptureHealthWidget movido para o rodapé                |
| 7814e92 | Mobile    | Ajustes responsivos em Admin e Publicações              |
| 3c2e90c | Branding  | Fonte Jura no wordmark Flow                             |
| 60c8026 | Branding  | Jura no wordmark Flow + aumento no header               |
| 8c8fcc0 | Templates | Aplica overrides de classificação no grid de cobertura  |

## 31. Versão 1.5 — Abril/2026

A v1.5 é o maior ciclo até aqui. Estreia um módulo inteiramente novo — Agendar Prazos Processuais — que eleva o DunaFlow de orquestrador de tarefas reativas (publicações, tratamento) para gestor proativo do prazo de resposta inicial do escritório, com classificação IA sem mapeamento Legal One, HITL (human-in-the-loop) para casos sensíveis e aprovisionamento de prazo fatal baseado no CPC 25. No mesmo ciclo, o agendamento de tarefas a partir de Publicações ganhou checagem prévia de duplicatas no próprio Legal One, todos os sete filtros de Publicações passaram a aceitar multiseleção, a administração de Templates foi reescrita para escala (paginação, multiseleção de escritórios, subcategoria livre, cobertura colapsável por categoria) e a integração Thomson Reuters no Tratamento Web foi estabilizada para o novo fluxo de login em dois passos com OnePass SSO.

### Destaque

*Com a v1.5, o DunaFlow deixa de ser só "o que chegou do Diário" e passa a cobrir também "o prazo que começa quando a carteira entra" — os dois lados do relógio processual do escritório.*

### 31.1. Novo módulo: Agendar Prazos Processuais

Este é o release principal da v1.5. O módulo trata o fluxo que acontece quando uma nova carteira é recebida (migração ou novo cliente) e o escritório precisa agendar os prazos iniciais de resposta — antes mesmo de o processo ter movimentações capturadas pelo DJEN. A fase de intake foi entregue em ondas, integrando a petição inicial (PI), taxonomia de tipos de pedido, classificação IA e aprovisionamento automático de prazo fatal.

O módulo foi construído em fases nomeadas por bloco (Fase 1 → Onda D → Bloco D2 → Bloco E → Bloco F), cada uma publicada separadamente para permitir validação incremental do time jurídico.

| Fase                                                   | O que entrega                                                                                                                                                                            |
|--------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Fase 1 — Backbone de intake</b>                     | Tabela principal de intakes, rotas de criação e consulta, vínculo com a carteira do escritório. Base para as fases seguintes.                                                            |
| <b>UI mínima + permissões</b>                          | Primeira tela de listagem e detalhe de intakes, atrás da permissão <code>can_use_prazos_iniciais</code> — acesso controlado por perfil.                                                  |
| <b>Fase 3a — Infra de classificação IA</b>             | Classificador dedicado ao módulo, sem depender de mapeamento do Legal One. Entende PI como texto e retorna tipo/subtipo/natureza.                                                        |
| <b>Fase 3b — Templates por (tipo, subtipo, office)</b> | Tabela <code>prazo_inicial_task_templates</code> . O template certo é escolhido por combinação de tipo + subtipo + escritório; fallback para o template global quando não há específico. |
| <b>Fase 3c — Natureza processual e produto</b>         | Campos <code>natureza_processo</code> e <code>produto</code> , além de <code>natureza_aplicavel</code> para refinar a classificação e o template sugerido.                               |
| <b>Fase 4a — Admin UI de templates</b>                 | Tela de administração dos templates: criar, editar, desativar, visualizar cobertura por (tipo, subtipo, office).                                                                         |
| <b>Fase 4a.2 (pin005)</b>                              | Infra de legacy + scheduling WIP + documentação inicial do módulo no admin. Sidebar ganhou consistência de rótulos.                                                                      |

|                                               |                                                                                                                                                                                                                |
|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Onda D — Limpeza e UX</b>                  | Refatoração geral da primeira versão, reset do circuit breaker da classificação IA, ajustes de UX e da fila de cancelamento.                                                                                   |
| <b>Observabilidade e robustez</b>             | Logs de estágios, métricas de latência da classificação IA, UI da fila de cancelamento com ações de reexecutar e abortar.                                                                                      |
| <b>Taxonomia humanizada + tipos de pedido</b> | Labels humanizados para cada item da taxonomia. Nova tabela prazo_inicial_tipos_pedido com seed de 46 tipos canônicos (contestação, impugnação, exceção, embargos etc.).                                       |
| <b>Prazo fatal com fundamentação</b>          | Cada intake passa a ter um prazo fatal calculado e uma fundamentação textual (base legal + prazo aplicável). Casos de agravo trazem o CNJ de origem.                                                           |
| <b>Bloco D2 — Extração de pedidos da PI</b>   | Extração automática dos pedidos contidos na petição inicial. Cada pedido recebe um provisionamento de prazo conforme CPC art. 25.                                                                              |
| <b>Bloco E — Agregados globais + CPC 25</b>   | Agregados globais por intake (total de pedidos, por tipo, alçada total) e aplicação do CPC 25 por pedido individualmente.                                                                                      |
| <b>Bloco F — Reanalisar + export XLSX</b>     | Botão de reanalisar intake (reprocessar PI sem criar intake novo) e export XLSX com agregados, pedidos e prazos fatais — insumo direto para o backoffice jurídico.                                             |
| <b>HITL expandido</b>                         | Tela de revisão humana (human-in-the-loop) com visão de pedidos, provisionamento por pedido, informação de agravo, botão de reanalisar e export embutido. Fecha o ciclo de intake sem o operador sair da tela. |

#### Decisão técnica

*A classificação IA foi desacoplada do Legal One de propósito — a PI entra no DunaFlow antes do processo estar cadastrado no L1, e exigir um lookup externo atrasaria o início do prazo. A natureza e o template vêm da própria IA; o ajuste humano acontece no HITL.*

### 31.2. Publicações — checagem de duplicata no Legal One antes de agendar

Um problema recorrente na v1.4 era o operador agendar uma tarefa para uma publicação cuja tarefa equivalente já existia no Legal One — duplicidade criada por contagem de prazo em fluxos paralelos (operador A, operador B, RPA antigo). A v1.5 blinda esse caso consultando o L1 antes de confirmar o agendamento.

- **Checagem prévia no /Tasks do L1:** ao clicar em "agendar", o modal faz uma consulta ao endpoint /Tasks do Legal One buscando por CNJ + tipo/subtipo + janela temporal. Se houver tarefa equivalente, o operador é bloqueado antes de criar duplicata.
- **Overlay bloqueante + banner vermelho forte:** enquanto a checagem roda, o modal mostra overlay bloqueante (impede clique duplo). Se encontra duplicata, mostra banner vermelho forte com o id e título da tarefa existente — o operador precisa confirmar explicitamente para prosseguir.

- **Respeita limite top=30 do /Tasks:** o endpoint /Tasks do Legal One limita a 30 resultados por consulta. A v1.5 respeita isso e, quando necessário, pagina a checagem — evita falsos negativos em carteiras com muitos cadastros.
- **publishDate + status.id + originOfficeId + linked\_office\_id:** o payload enviado ao L1 passou a carregar todos os defaults obrigatórios. O 422 (validação do L1) que ocorria em alguns agendamentos foi eliminado.

### 31.3. Publicações — multiseleção em todos os filtros

Até a v1.4, os filtros de Publicações eram de valor único (um status, um escritório, uma categoria). A v1.5 estende multiseleção a todos os sete filtros, alinhando o módulo ao comportamento já esperado por operadores acostumados a triar em lote.

- Status e escritório — primeiros a virar multiselect, a pedido direto da operação (triagem por vários lotes simultâneos).
- Categoria, UF, polo, natureza e vínculo — ampliação para os cinco filtros restantes no mesmo padrão, com badges de "filtros ativos" no desktop.
- Reaplicar templates agora é escopado aos filtros ativos — se o operador filtrou "apenas sem processo + UF:BA", o reprocessamento roda só nesse subconjunto. Contador de publicações afetadas aparece na paginação antes da confirmação.
- Combobox com busca para subtipo — o campo subtipo do modal de agendar vira combobox com busca por texto, acelerando a seleção em taxonomias grandes.
- Busca por CNJ + autoria do AGENDADO — operador consegue buscar publicação por CNJ diretamente e ver quem agendou cada item no histórico.

### 31.4. Publicações — resiliência da busca em andamento

Buscas longas e concorrentes começaram a esbarrar em casos de borda que a v1.4 não cobria. A v1.5 trata três deles.

| Caso                                                     | Mitigação                                                                                                                                                       |
|----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Busca travada em estado intermediário</b>             | Cancelamento cooperativo — operador pode cancelar busca em andamento, e o backend libera recursos gracefully, sem deixar registros meio-classificados no banco. |
| <b>UniqueViolation no meio de um batch</b>               | Deteção por id canônico dentro do próprio batch — duplicatas intra-batch são filtradas antes do flush, evitando rollback da transação inteira.                  |
| <b>Buscas órfãs (processo morto deixou status ATIVA)</b> | Watchdog periódico varre buscas em estado ATIVA sem worker vinculado há N minutos e marca como ERRO — libera o usuário para disparar nova busca.                |

### 31.5. Templates — administração mais escalável

O módulo de Templates passa por um amadurecimento completo, transformando de "lista simples" em interface de administração capaz de sustentar o crescimento do catálogo.



- Responsável opcional no template — antes era obrigatório; agora o template pode ser criado sem responsável fixo, forçando seleção manual no modal de criar tarefa (trava de validação).
- Subcategoria livre — em casos em que a taxonomia oficial ainda não contempla o caso, o operador pode escrever subcategoria livre, mantendo rastreabilidade.
- Multiseleção de escritórios — um mesmo template pode ser associado a múltiplos escritórios, reduzindo duplicação no catálogo.
- Paginação na listagem + "topo da lista" no modal — lista agora pagina, e o modal oferece atalho para voltar ao topo.
- Coverage com collapse por categoria — grid de cobertura passou a colapsar por categoria para permitir visão geral em catálogos grandes, sem perder a capacidade de drill-down.
- Asterisco no segundo modal — marca visual em campos que são herdados de outra classificação, reduzindo confusão sobre a origem do valor.

### 31.6. Tratamento Web — estabilização do login Thomson Reuters

A integração de Tratamento Web com o Legal One passou por uma atualização de login do lado da Thomson Reuters — OnePass SSO, fluxo em dois passos. A v1.5 reescreve a parte de autenticação do RPA para acompanhar.

- Login em dois passos — o novo fluxo do OnePass requer e-mail + continue + senha. A v1.5 detecta a separação e preenche na ordem correta.
- Auth handoff — quando o TR faz redirect interno para o domínio do OnePass, o RPA acompanha o handoff e mantém a sessão íntegra.
- Abort do single-page login em auto-redirect — antes o RPA tentava preencher o formulário de login single-page mesmo quando o TR já tinha redirecionado; agora aborta a tentativa e aguarda a página certa.
- Trigger OnePass redirect on blur — alguns elementos só disparam o redirect ao perder foco; a v1.5 força blur explícito para avançar.
- Logs no stdout — toda a sequência de login passa a logar por stdout (além do arquivo de log), facilitando diagnóstico em produção.

### 31.7. Legal One client — lookup de CNJ tolerante

O Legal One aceita CNJ em dois formatos: masked (com pontos e traços, ex: 0001234-56.2024.8.26.0001) e digits-only (somente números). A v1.5 torna o lookup do cliente interno tolerante aos dois — se o primeiro formato falha, tenta o segundo automaticamente.

Esse fix destrava integrações onde o CNJ era salvo em um formato e a busca era feita no outro, devolvendo 404 de forma silenciosa.

### 31.8. Dashboard de operação de Publicações — refinós

- "Refine publications operations dashboard" — ajustes de legibilidade, ordenação e agrupamento de cards do dashboard operacional (não o executivo da v1.4, e sim o operacional de Publicações, com foco em triagem ativa).
- Fila de tratamento — correção de comportamento quando o monitor web perdia contato com a fila, passando a travar o painel.
- Monitor de tratamento web sem travamento — guard contra bloqueio do monitor em caso de crawl longo.

### 31.9. Changelog detalhado — commits relevantes

| Hash    | Tema                     |
|---------|--------------------------|
| cbf648c | Prazos Processuais       |
| 8e87652 | Prazos Processuais       |
| 1aeba3a | Prazos Processuais       |
| f75bf0b | Prazos Processuais       |
| d5e97cf | Prazos Processuais       |
| 42d6984 | Prazos Processuais       |
| 4e31bbf | Prazos Processuais       |
| 87b3a88 | Prazos Processuais       |
| aba0398 | Prazos Processuais       |
| 4d5987d | Prazos Processuais       |
| f48d577 | Prazos Processuais       |
| 0ed5c92 | Prazos Processuais       |
| 4d5f455 | Prazos Processuais       |
| 6178445 | Prazos Processuais       |
| b5f9e27 | Prazos Processuais       |
| 55531a5 | Prazos Processuais       |
| 20cc2b7 | Prazos Processuais       |
| ec889e5 | Prazos Processuais       |
| fcd6dc1 | Prazos Processuais       |
| a7a0ab7 | Publicações / Duplicatas |
| 7af0f5d | Publicações / Duplicatas |
| 5224f79 | Publicações / Duplicatas |
| 3ce5336 | Publicações / Legal One  |
| 83b169a | Publicações / UX         |

|                |                          |
|----------------|--------------------------|
| <b>d897b3b</b> | Publicações / Legal One  |
| <b>5a5e9f4</b> | Publicações / UX         |
| <b>1769ccf</b> | Publicações / Legal One  |
| <b>10e449a</b> | Publicações / UX         |
| <b>4f8a6c4</b> | Publicações / UX         |
| <b>4e814c2</b> | Publicações / Filtros    |
| <b>65193d5</b> | Publicações / Filtros    |
| <b>424fb59</b> | Publicações / Busca      |
| <b>96480a4</b> | Publicações / Busca      |
| <b>0cb587b</b> | Publicações / Dashboard  |
| <b>93704da</b> | Publicações / Tratamento |
| <b>d679434</b> | Publicações / Tratamento |
| <b>7826e3a</b> | Templates                |
| <b>d829283</b> | Templates                |
| <b>4a09655</b> | Templates                |
| <b>8c9cb8d</b> | Tratamento Web           |
| <b>a8f34af</b> | Tratamento Web           |
| <b>05796e1</b> | Tratamento Web           |
| <b>ee44228</b> | Legal One                |
| <b>3d8bd3e</b> | Legal One client         |

## 32. Versão 1.6 — Abril/2026

A v1.6 é, em escopo, o maior release já feito no DunaFlow — mais até do que a v1.5. Enquanto a v1.5 acrescentou um módulo novo (Prazos Processuais, antes chamado Prazos Iniciais), a v1.6 acrescenta quatro módulos completamente novos ao mesmo tempo e incorpora, dentro do Prazos Processuais, um submódulo dedicado à Análise Recursal — o que justificou o próprio rename do módulo. Os quatro módulos novos cobrem quatro problemas diferentes do escritório: recebimento e resposta às DMIs do Banco do Brasil (OneRequest), gestão de desempenho por equipe com capacity planning (Minha Equipe), redistribuição AO VIVO de carga entre colaboradores no Legal One (Balanceador de Agenda) e emissão de parecer sobre viabilidade recursal a partir do PDF do processo (Análise Recursal). Os módulos existentes também evoluíram: Publicações ganhou dropdown de detalhamento por escritório responsável e relatório crítico de performance; a plataforma como um todo teve refino de sidebar, permissões condensadas e desacoplamento do sync do Legal One do fluxo de login. O rename de Prazos Iniciais para Prazos Processuais reflete o novo escopo do módulo, que agora cobre também o pós-julgamento.

## Contexto

A v1.6 marca o momento em que o DunaFlow deixa de ser exclusivamente sobre publicações e prazos iniciais e passa a ser uma plataforma de operação jurídica ampla — recebendo demandas externas (DMIs), medindo desempenho interno, redistribuindo carga e emitindo pareceres. Os módulos foram construídos em paralelo pela mesma base (FastAPI + Legal One + Sonnet/Haiku), reusando infraestrutura de classificação IA, jobs persistentes e integrações RPA.

### 32.1. Novo módulo: OneRequest — orquestração das DMIs do Banco do Brasil

O OneRequest resolve um problema recorrente da carteira Banco do Brasil: as Demandas de Manifestação de Interesse (DMIs) chegam por diferentes canais, seguem tratamento manual em planilha e o vínculo com a tarefa correspondente no Legal One é feito de forma dispersa. O novo módulo centraliza intake, tratamento, agendamento e auditoria de todas as DMIs em uma única tela, com alertas proativos para vencimentos e drill-through para os casos.

- **Intake das DMIs BB + tela de tratamento + agendamento no L1 (fase 1):** primeira entrega do módulo. Lista de DMIs recebidas, tela de tratamento com sugestão de setor e prazo, botão para agendar tarefa no Legal One diretamente da DMI e permissão `can_use_onerequest` para gate por perfil.
- **Sync read-only Postgres da fonte + advisory lock:** job horário lê a base de origem (RPA que ingere as DMIs) e espelha em `onr_solicitacoes`. O DunaFlow é dono do tratamento; a fonte só entrega os registros novos. Advisory lock do Postgres evita duplicate key e trabalho 4x quando há múltiplos workers do uvicorn.
- **Sync espelha tratamento na transição:** quando uma DMI já vem com responsável, setor, data e anotação preenchidos na fonte, o sync espelha esses campos no primeiro import, evitando zerar o tratamento existente. Flag `onerequest_sync_espelha_tratamento` controla o comportamento (True na transição, False depois).
- **Trava-duplo no Agendar:** antes de agendar tarefa a partir da DMI, o módulo verifica se já existe uma tarefa PENDENTE no L1 para essa mesma DMI. Se existir, avisa e pede confirmação. Cumprida não bloqueia (é o próprio ciclo natural).
- **Alerta vence-hoje pelo Teams — Webhook e Graph:** duas implementações complementares. Webhook (Power Automate) via `TEAMS_ALERT_WEBHOOK_URL` para canais corporativos M365. Microsoft Graph via MSAL delegado — envia DM direto no nome da operadora responsável, ativado por `TEAMS_ALERT_ENABLED`. Feature-flags permitem escolher canal ou desativar em ambientes homologação.
- **Auditoria total + auto-refresh L1 + status L1 sob demanda:** cada DMI carrega histórico completo (quem tratou, agendou, concluiu, com timestamps). Auto-refresh horário sincroniza status das tarefas do L1 (play/stop no header). Botão "Acompanhar" na aba Agendadas força atualização sob demanda.

- **KPIs, dashboard dedicado e drill-through:** cards clicáveis (chip de ícone colorido por farol, número tabular, hover com elevação) filtram a tabela por farol; dashboard próprio com série de recebimento/agendamento, farol de vencimento e carga por responsável; clicar em qualquer KPI, farol, setor ou responsável abre a tela de tratamento já filtrada.
- **Filtros de data + aba Concluídas + export Excel:** filtro por data de disponibilização e prazo fatal; aba Concluídas com badge (respondida/arquivada); export Excel respeitando os filtros ativos. Modal de tratar mais largo, coluna Polo (ativo/passivo), coluna Disponibilizada (recebido\_em).
- **Tarefa avulsa sem pasta quando o processo não está no L1:** quando a DMI se refere a processo ainda não cadastrado, o módulo agenda tarefa avulsa (create\_task sem vínculo, escritório derivado do polo/setor). Verificação proativa da existência do processo no L1 roda hora em hora (badge, botão, filtro); migration onr004.
- **Busca vira modal de auditoria + polish UX:** a aba Busca foi transformada em modal de auditoria — cross-status entre todas as DMIs do mesmo processo (com quem agendou, quando, quando concluiu). Barra de status condensada (ingestão + auto-atualização L1 + abertas numa linha), busca com ícone embutido, zebra striping na tabela.
- **Card de DMIs totais pendentes na Home:** a fileira de KPIs da Home ganhou um card estático mostrando o total de DMIs abertas (não clicável, estilo distinto: tint primary + ícone Inbox). O grid foi ampliado para 6 colunas.

## 32.2. Novo módulo: Minha Equipe — desempenho por equipe

O módulo Minha Equipe traz para o DunaFlow o que a MDR já fazia em planilhas dispersas: acompanhar carga, vazão, jornada e capacidade individual e coletiva das equipes de contencioso. O piloto começou com BB Réu e evoluiu para multi-time (11 equipes: 6 Contencioso Passivo + 5 Recuperação de Crédito). O módulo é alimentado pelo relatório Agenda Analytics do Legal One, ingerido de forma automática todos os dias e complementado por um mecanismo próprio de cancelamento em massa de tarefas duplicadas.

| Componente                                                    | O que faz                                                                                                                                                                                                                                                            |
|---------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Painel de vazão + pool de atraso + jornada + top tipos</b> | Quatro gráficos principais: vazão de conclusões (semanal e mensal), pool de tarefas atrasadas em aberto, curva de jornada típica e ranking dos tipos que mais consomem tempo.                                                                                        |
| <b>Raio-X individual</b>                                      | Modal por colaborador com passado (tarefas concluídas, tempo médio, aderência ao prazo) e futuro (backlog, próximas por prazo). Ganhou jornada típica (horário de chegada/saída pelas conclusões) e tempo de decisão por tipo (mediana do intervalo entre entregas). |
| <b>Export Excel por recorte</b>                               | Cliques em qualquer barra/fatia dos gráficos abrem confirmação de export com o recorte específico (ex: atrasadas por tipo X, na pessoa Y) contendo o número do processo — insumo direto para campanha focada.                                                        |
| <b>Relatórios PDF (setor e individual)</b>                    | Emissão de relatório PDF por setor ou por pessoa, com narrativa Sonnet + fallback determinístico. Roda como job persistente no servidor (migration                                                                                                                   |

|                                                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                      | perf003, tabela perf_relatorio) — sobrevive a navegação, listagem com polling, botão de download.                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Ingestão automática (Agenda Analytics)</b>        | Worker 9h-12h30 BRT (30/30min) baixa o relatório do dia via cookie .ASPXAUTH existente. Retry até o relatório aparecer. Botão "Atualizar agora" + status no header. Fallback pela API de ingest quando o download falha. Aposenta o seed manual.                                                                                                                                                                                                                                                         |
| <b>Ingest atômico</b>                                | Todo o seed é uma única transação (delete + inserts + commit único no fim). Kill no meio faz rollback e preserva os dados antigos completos; antes deixava a tabela parcial (78k de 224k).                                                                                                                                                                                                                                                                                                               |
| <b>Relatório sob demanda via Playwright</b>          | Novo runner generate-report.js loga no L1 real e clica em "Gerar" no navegador — o POST headless não conclui porque a página "Gerando" depende de SignalR. O clique dispara job server-side que completa (~30-45s) mesmo com o browser já fechado. Jobs APScheduler em 4h e 13h + download 9h-12h30 de fallback. Validado end-to-end: 224.744 tarefas, geração em ~30s.                                                                                                                                  |
| <b>Multi-time / setores</b>                          | Parser multi-aba da planilha de squads (equipe + is_supervisor por pessoa), migration perf004; 6 times originais, +5 setores Autor (Recuperação de Crédito) no registro. 11 times, 117 pessoas, 190k tarefas. Sidebar com dois grupos: Contencioso Passivo e Recuperação de Crédito, expansíveis.                                                                                                                                                                                                        |
| <b>Editor de equipe (CRUD roster)</b>                | Modal "Ajustar equipe" com CRUD do roster — editar cargo/time/supervisor/ativo, mover entre times, adicionar (busca no catálogo de usuários do L1), excluir do sistema com confirmação (desvincula tarefas, preserva histórico). Ordenado supervisor > advogado > assistente > estagiário.                                                                                                                                                                                                               |
| <b>Board "Tarefas mais importantes" customizável</b> | Antes o board mostrava top-12 automático por volume. Agora o supervisor fixa/remove subtipos via "Editar tarefas" (combobox do catálogo do time). Sem curadoria, mantém top-12 automático. Indicador "seleção fixa" vs "top-12 automático" no header. Análise de capacity por tipo (tempo de conclusão + tempo de trabalho por tarefa) e preview de tarefas duplicadas para cancelar.                                                                                                                    |
| <b>Cancelamento de duplicadas em massa</b>           | Rotina noturna (4h) gera pool fresco e cancela duplicadas de TODAS as carteiras (mesma pasta + subtipo, mantém a mais antiga), mas SÓ dos subtipos da whitelist. Whitelist editável via UI admin ("Duplicadas") com combobox do catálogo, ativar/desativar/remover. Cancelamento em lote pré-check via OR filter id eq (chunks de 28), POST em lote, verify em lote com 1 retry. Corta de ~9 min para dezenas de segundos por rodada. Auditoria total (perf_cancel_massa_log) com breakdown por subtipo. |

### 32.3. Novo módulo: Balanceador de Agenda

Redistribuição AO VIVO de carga entre colaboradores do Legal One. Antes, quando um advogado saía de férias ou uma pessoa nova entrava na equipe, o supervisor precisava abrir tarefa a tarefa no L1 e trocar responsável manualmente. O Balanceador puxa as tarefas ao vivo (não do snapshot), permite arrastar em quantidade ou individualmente, escolher entre destinos internos (roster do setor) ou externos (busca no catálogo do L1) e aplica a reatribuição real via PATCH participants.

- Diagnóstico de carga por colaborador — matriz por pessoa mostrando atrasadas, com prazo fatal hoje, futuras. Filtro por cargo, expansão para ver detalhe (descrição, prazo) por tarefa. Barra de progresso durante o load ao vivo.
- Redistribuição AO VIVO — puxa pendentes por colaborador via participants + statusId, respeita orderby por prazo mais urgente, teto de páginas + total real por count para não estourar rate limit. subTypeId → nome pelo catálogo local. Detalhe reusa as tarefas ao vivo com descrição real.
- Distribuição em fila (round-robin) — supervisor seleciona subtipo e um conjunto de destinos, define "quantas mover" ou "mover todas", e o sistema rateia em fila entre os destinos escolhidos. Multiselect no combobox. Alvo de fila só recebe (sem carregar as tarefas dele) — economia de chamada.
- Reatribuição real via PATCH participants — troca responsável + executante preservando solicitante. Job server-backed com progresso por polling + buckets no overlay. Workflow cai em bucket bloqueado (API trava esse tipo, só RPA/UI). Dry-run "Simular" lê sem gravar. Migration perf010.
- Sidebar aninhado — macro "Minha Equipe" desdobra em sub-grupos Contencioso Passivo (6 times) e Recuperação de Crédito (5 times); cada nível é recolhível. Consistência entre o sidebar e a árvore de permissões do admin.
- Fila com destinos recorrentes — o sistema aprende por origem + subtipo os destinos mais usados e sugere no topo já marcados (badge "recorrente"). Tabela balanceador\_fila\_pref, migration bal002.
- Reverter tudo + log persistente — botão "Reverter" desfaz a última redistribuição. Log completo na aba Relatórios (balanceador\_log, migration bal001).
- Recorte por período + destinos externos — filtro de deadLine (ge/le) feito diretamente no L1, com toggle "incluir atrasadas" (corrige o teto que antes comia as atrasadas e escondia o futuro). Combobox de busca no catálogo /usuarios do L1 permite mandar tarefa para gente fora do setor, misturando com os do roster.
- UI da redistribuição destravada — modais aninhados com pointer-events:auto explícito para sobrepor o pointer-events:none que o Radix deixa preso no body ao fechar Dialog aninhado. Dialogs aninhados com modal={false} (não deixam o pai inert ao fechar); overlay de progresso portalado no body; modal principal ignora clique-fora.

### 32.4. Novo módulo: Análise Recursal (dentro de Prazos Processuais)

Este submódulo é o que motivou o rename Prazos Iniciais → Prazos Processuais. Enquanto o Prazos Iniciais original tratava de intake da PI e agendamento de contestação, a Análise Recursal cobre o outro extremo do ciclo: quando o processo já foi julgado, o operador faz upload do PDF completo e o DunaFlow gera um veredito de viabilidade recursal + parecer formatado (pronto para o advogado copiar e enviar como assunto + corpo) usando Sonnet em batch. O custo do preparo do recurso é determinístico via tabela de custas dos 27 TJs.

| Componente                                         | O que faz                                                                                                                                                                                                                                                                                                                                                                        |
|----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Upload de PDFs de processo</b>                  | Operador manda o PDF completo pela tela do módulo. O DunaFlow reusa a extração mecânica dos Prazos Processuais (pdfplumber) para levantar decisão, petição inicial e documentos juntados.                                                                                                                                                                                        |
| <b>Veredito recursal (Sonnet)</b>                  | Uma chamada Sonnet em batch por processo. Retorna: probabilidade de reversão (alta/média/baixa), recursos possíveis (incluindo Recurso Inominado para JEC, +10 dias úteis, além de apelação/agravo/resp/re), pontos de atenção, fundamentação do juiz e parecer formatado como assunto + corpo pronto para copiar.                                                               |
| <b>Regras de domínio do Master</b>                 | Prompt embute as regras próprias do Master (banco): (1) nunca recomenda embargos de declaração; (2) regra geral é NÃO recorrer, só foge disso com fundamento forte; (3) agravo só se recorre em dano irreversível (desaverbação/cancelamento de operações em liminar); (4) campo crítico contestacao_com_documentos (presença, não qualidade) — badge no card, migration rcr004. |
| <b>Custas dos 27 TJs (determinístico)</b>          | Tabela seed com custas de recurso dos 27 tribunais estaduais, validada contra exemplos R\$ 10k / R\$ 50k. Cálculo do preparo é determinístico — não passa pela IA. Migration rcr002 (seed custas).                                                                                                                                                                               |
| <b>Prazo fatal determinístico (+15 dias úteis)</b> | A IA extrai a data de intimação/publicação da decisão; o código calcula prazo fatal = +15 dias úteis (5 p/ embargos de declaração) via prazo_calculator do próprio Prazos Processuais (respeita feriados nacionais). Coluna data_intimacao, migration rcr003. Validade: intimação 30/06 + 15 úteis = 21/07/2026, igual ao cálculo manual do sócio.                               |
| <b>Pontos de atenção (caixa âmbar)</b>             | Quando o diagnóstico não é seguro (confiança média/baixa, íntegra truncada, decisão ambígua, dado faltando, recorrer limítrofe), a IA lista dicas INCISIVAS e acionáveis para o advogado revisar. Guia INTERNO — não vai no parecer entregue ao cliente. Coluna pontos_de_atencao, migration rcr005.                                                                             |
| <b>Fidelidade do parecer</b>                       | Fatiamento inteligente da íntegra (documentos recentes + petição inicial, cap por documento) em vez de cortar cego em 30k tokens. CPF injetado deterministicamente rankeado por frequência (o do autor recorre dezenas de vezes vs 1x de anexos). Número limpo (CNJ) no assunto/corpo em vez do nome do arquivo. Fundamentação do juiz como continuação em minúscula.            |
| <b>Fire-and-forget worker</b>                      | Worker de background (APScheduler, 3 min) auto-submete os PDFs subidos e auto-aplica os vereditos quando o batch termina — o operador não precisa ficar na tela nem clicar "Atualizar". A página se auto-atualiza (poll 20s) enquanto há processo em andamento. Setting recursal_auto_worker_enabled.                                                                            |
| <b>Barra de progresso da Análise Recursal</b>      | Endpoint /progresso conta por status (aguardando/em análise/analizado/erro/sem texto) e a barra aparece enquanto há processo em jogo. Auto-poll global (não só na página atual).                                                                                                                                                                                                 |
| <b>OCR direcionado para documentos-imagem</b>      | Tesseract roda SÓ nas páginas sem texto (as que o pdfplumber não leu por serem escaneadas). Capado em 15 páginas / 18k caracteres. Via threadpool no upload para não travar o event loop. Texto recuperado entra em seção própria na mensagem para a IA. Dockerfile + requirements: tesseract-ocr + por + pymupdf + pytesseract.                                                 |
| <b>Detecção de página-imagem por</b>               | O PJe carimba um banner em toda página, então até a escaneada tem                                                                                                                                                                                                                                                                                                                |



|                                                 |                                                                                                                                                                                                                                                                                                   |
|-------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>cobertura</b>                                | ~20-350 chars — o limiar antigo (20) pulava tudo. Novo: página com <600 chars E imagem cobrindo >40% da folha = escaneada → OCR. Validado no processo real: achou as 5 páginas escaneadas e leu o comprovante COMPEA do autor.                                                                    |
| <b>Manifesto de documentos-imagem sem texto</b> | Para não perder a existência de prova documental em imagem, a mensagem para a IA inclui manifesto de TODOS os documentos juntados (label + data), marcando os que são imagem sem texto. O prompt instrui a IA a registrar em pontos_de_atencao quando houver documento-imagem relevante não lido. |

### Rename do módulo

Com a incorporação da Análise Recursal, o módulo antes chamado "Prazos Iniciais" foi renomeado para "Prazos Processuais" — reflete o novo escopo, que cobre desde a chegada da PI (intake inicial) até o pós-julgamento (análise recursal). O item da sidebar foi renomeado consistentemente. Nomes de tabela (prazo\_inicial\_tipos\_pedido, prazo\_inicial\_task\_templates) e a permissão can\_use\_prazos\_iniciais foram preservados por compatibilidade — só o rótulo humano mudou.

## 32.5. Prazos Processuais — evoluções pós-rename

- Upload de petição inicial em LOTE + arquivamento em lote — o intake ganhou modo em lote para receber várias PIs de uma carteira nova de uma vez. Arquivamento em lote no final.
- Detecção de despacho de citação (sinal-âncora) — quando o DunaFlow encontra o despacho de citação no texto, ele troca patrocínio pela flag neutra vinculada\_master (evita etiquetagem prematura do lado do processo).
- Remove fluxo de devolução automática + conceito is\_devolucao do AJUS — o fluxo antigo de devolução, herdado do AJUS, foi removido. O intake é sempre linear agora (sem loop de devolução), simplificando a UX e o modelo de dados.
- Header não espreme o título nos botões — quando o header ganhou botões extras (upload em lote, arquivamento em lote), o layout empurrava o título; agora quebra linha sem espremer.
- Remove consumidor órfão de devolução no PrazosIniciaisPage — resquício da remoção do fluxo de devolução. Cleanup interno.
- JSONB → JSON no SQLite (com with\_variant) — destrava o fixture db\_session usado nos testes automáticos; JSONB é específico do Postgres.

## 32.6. Publicações — dropdown de detalhamento e relatório crítico

- **Dropdown de detalhamento por escritório responsável no Tratamento por operador:** linha da tabela do Tratamento por Operador ganhou chevron expansível. Ao abrir, mostra breakdown lazy por operador (agrupado por linked\_office\_id) usando o path do catálogo. Escritório fora do catálogo vira "Escritório #id" (distinto de "sem vínculo"). Endpoint

publications-operator-offices agrupa agendadas + ciências nas mesmas janelas/colunas da tabela pai.

- **Relatório crítico de performance (admin):** PDF server-side via Chromium (Playwright) por período mínimo de 5 dias. Diagnóstico Sonnet com fallback determinístico para o caso de a IA falhar/timeout. Disparado do dashboard de Publicações (admin only).
- **Serializa Decimal e date no payload do Sonnet:** json.dumps padrão do Python quebra em Decimal e date. O relatório crítico caía sempre no fallback determinístico por causa disso. Fix serializa antes de mandar para a IA.

### 32.7. Admin, Auth e plataforma — refinós

- Permissões condensadas em dropdown por usuário — antes eram 5 colunas na tabela de usuários (uma por permissão). Agora um dropdown de checkboxes por linha, salva na hora, badges das ativas no botão. Edit-mode passa a cuidar só de papel + escritório.
- Sidebar com seções recolhíveis — clique no título abre/fecha, com chevron. Estado persistido em localStorage. Todas as famílias (Publicações, Prazos Processuais, Minha Equipe, LegalOne, Admin) respondem.
- Desacopla sync do Legal One do login — antes o login esperava o sync inicial do L1, que às vezes travava e derrubava a tela. Agora /me devolve as permissões do banco imediatamente e o sync L1 acontece em background.
- Layout min-w-0 nas telas — corrige estouro horizontal em telas estreitas. Logo DunaFlow com shrink-0 para não espremer. OneRequest pré-preenche setor/responsável/data da sugestão (alinha rótulo BB Execução e Encerramento). Abas roláveis.
- Modal de tratamento sem scroll horizontal — overflow-x + quebra nas tarefas do L1. Caixa do texto da DMI maior/legível.
- Logo DUNATECH da sidebar agora rola com o menu — antes ficava fixa no fundo da viewport, criava sensação de menu "quebrado" quando o usuário rolava. Agora acompanha.
- Sidebar sem flex-grow no footer — logo flui após o menu em vez de grudar no fundo. Sinergia com o item anterior.
- "Cancelar Duplicadas" no menu LegalOne — a página de gestão de whitelist de duplicadas saiu da aba Admin e virou item dedicado no grupo LegalOne do sidebar (/cancelamento-duplicadas), gate isAdmin. Cabe melhor mentalmente lá — é uma operação sobre o L1, não um ajuste do sistema.

### 32.8. Changelog detalhado — commits relevantes

| Hash    | Tema       |
|---------|------------|
| d32ab97 | OneRequest |
| a06dfd2 | OneRequest |
| b3129a9 | OneRequest |

|                |              |
|----------------|--------------|
| <b>d310cd2</b> | OneRequest   |
| <b>5cbbbb5</b> | OneRequest   |
| <b>cf0a642</b> | OneRequest   |
| <b>acdf763</b> | OneRequest   |
| <b>ce9c746</b> | OneRequest   |
| <b>73caebf</b> | OneRequest   |
| <b>74bea62</b> | OneRequest   |
| <b>2940263</b> | OneRequest   |
| <b>e1cae30</b> | OneRequest   |
| <b>57b534c</b> | OneRequest   |
| <b>967bf10</b> | OneRequest   |
| <b>8e2c799</b> | OneRequest   |
| <b>021c517</b> | OneRequest   |
| <b>e8e36bc</b> | OneRequest   |
| <b>f936463</b> | OneRequest   |
| <b>debbc7a</b> | OneRequest   |
| <b>cff4c9a</b> | OneRequest   |
| <b>ad1d8e3</b> | OneRequest   |
| <b>139de4e</b> | Minha Equipe |
| <b>30b6367</b> | Minha Equipe |
| <b>d71529c</b> | Minha Equipe |
| <b>0c14c04</b> | Minha Equipe |
| <b>24ddb26</b> | Minha Equipe |
| <b>cfe1556</b> | Minha Equipe |
| <b>3e12f31</b> | Minha Equipe |
| <b>d40f2da</b> | Minha Equipe |
| <b>cff4c9a</b> | Minha Equipe |
| <b>19c9bc8</b> | Minha Equipe |
| <b>4d7d21c</b> | Minha Equipe |
| <b>7559bd9</b> | Minha Equipe |
| <b>90896e3</b> | Minha Equipe |
| <b>83ab1a4</b> | Minha Equipe |
| <b>e794200</b> | Minha Equipe |

|              |                    |
|--------------|--------------------|
| 938b4a8      | Minha Equipe       |
| 3a06a23      | Minha Equipe       |
| 375ac9c      | Minha Equipe       |
| 96a4ede      | Minha Equipe       |
| 713bc83      | Minha Equipe       |
| 1b7b41c      | Minha Equipe       |
| c67e7ef      | Minha Equipe       |
| 2c21e42      | Minha Equipe       |
| a020ca2      | Minha Equipe       |
| e3f9400      | Minha Equipe       |
| c14c506      | Balanceador        |
| 846694e      | Balanceador        |
| b011acf      | Balanceador        |
| 4eecee5      | Balanceador        |
| 536f183      | Balanceador        |
| a24d390      | Balanceador        |
| ff69c0c      | Balanceador        |
| cfb7b41c-ptr | Balanceador        |
| ab052b8      | Análise Recursal   |
| 030a6e2      | Análise Recursal   |
| 3ef0384      | Análise Recursal   |
| 38c1df5      | Análise Recursal   |
| 3b98286      | Análise Recursal   |
| d0e1e0a      | Análise Recursal   |
| a7d2f27      | Análise Recursal   |
| d942682      | Análise Recursal   |
| 0518edd      | Análise Recursal   |
| 6605350      | Prazos Processuais |
| 98a73d7      | Prazos Processuais |
| 7a37e13      | Prazos Processuais |
| a0336c1      | Prazos Processuais |
| 6004ad5      | Prazos Processuais |
| eb14b98      | Testes / DB        |

|         |             |
|---------|-------------|
| 089111d | Publicações |
| 9f26746 | Publicações |
| a07ea5f | Publicações |
| 371a08f | Admin       |
| 55f6bd4 | UI          |
| 6d2d763 | Auth        |
| 65df5eb | Layout      |
| cf9ca3c | Layout      |
| 3f8edc3 | Layout      |
| 95d174e | Layout      |

## Apêndices

### Apêndice A — Glossário

| Termo                 | Significado                                                                          |
|-----------------------|--------------------------------------------------------------------------------------|
| Batch / Lote          | Conjunto de itens processados juntos (criação de tarefas, classificação, tratamento) |
| BatchExecution        | Registro mestre de um lote, com status, totais e timestamps                          |
| BatchExecutionItem    | Cada item individual dentro de um lote                                               |
| CNJ                   | Conselho Nacional de Justiça; mantenedor do DataJud                                  |
| DataJud               | Base pública do CNJ com dados processuais de todos os tribunais                      |
| DJEN                  | Diário de Justiça Eletrônico Nacional (CNJ)                                          |
| Comunica              | Plataforma de comunicações processuais oficiais do CNJ                               |
| Legal One             | CRM jurídico utilizado pela MDR                                                      |
| Onesid / Onerequest   | Parceiros/fontes de entrada de tarefas para criação em lote                          |
| Publicação            | Ato oficial publicado em diário; insumo para classificação e agendamento             |
| RPA                   | Robotic Process Automation; aqui via Playwright contra a UI Legal One                |
| Playwright            | Biblioteca de automação de navegador utilizada para o RPA                            |
| Strategy              | Padrão de projeto onde algoritmos intercambiáveis compartilham interface             |
| APScheduler           | Scheduler Python utilizado para automações agendadas                                 |
| Régua de maturidade   | Classificação em 7 níveis do estado do processo no ciclo de baixa                    |
| UF (via CNJ)          | Derivação da unidade federativa a partir do número do processo                       |
| Override manual       | Classificação feita por humano que não pode ser sobrescrita automaticamente          |
| Batch API (Anthropic) | Endpoint com 50% de desconto em troca de SLA assíncrono                              |

### Apêndice B — Endpoints principais (selecionados)

Todos os endpoints são versionados sob /api/v1 e exigem autenticação JWT (exceto /auth/login).

| Recurso | Método   | Rota                  | Função                          |
|---------|----------|-----------------------|---------------------------------|
| Auth    | POST     | /auth/login           | Autentica usuário e retorna JWT |
| Auth    | POST     | /auth/change-password | Altera senha do usuário logado  |
| Admin   | GET/POST | /admin/users          | Gestão de usuários              |
| Sectors | GET/POST | /sectors              | Setores do escritório           |
| Squads  | GET/POST | /squads               | Squads por setor                |

|                |          |                                     |                                         |
|----------------|----------|-------------------------------------|-----------------------------------------|
| Offices        | GET      | /offices                            | Escritórios Legal One sincronizados     |
| Tasks          | POST     | /tasks/batch                        | Criação em lote de tarefas (Strategy)   |
| Templates      | GET/POST | /task-templates                     | Modelos de tarefa reaproveitáveis       |
| Publications   | GET      | /publications/records               | Listagem de publicações                 |
| Publications   | POST     | /publications/records/reclassify    | Reclassificação manual inline           |
| Classifier     | POST     | /classifier/run                     | Dispara classificação de backlog        |
| Classifier     | GET      | /classifier/classification-taxonomy | Taxonomia de categorias e subcategorias |
| Treatment      | POST     | /publication-treatment/queue        | Enfileira publicações para RPA          |
| Treatment      | GET      | /publication-treatment/runs         | Histórico de execuções                  |
| Automations    | GET/POST | /automations                        | Jobs agendados                          |
| Dashboard      | GET      | /dashboard/home                     | Visão consolidada da home               |
| Capture Health | GET      | /capture-health                     | Sinais de saúde das capturas            |

A lista completa e atualizada está em `/api/v1/openapi.json` e é navegável via `/docs` (Swagger) e `/redoc`.

## Apêndice C — Taxonomia de classificação (resumo)

A taxonomia consolidada reflete a prática real do contencioso MDR. Abaixo, um resumo das categorias principais:

- 1º Grau — Cível / Execução
- 2º Grau — Cível
- Tutela
- Audiência Agendada
- Complementar Custas
- Manifestação das Partes
- Provas
- Sentença
- Recursos (Apelação, Embargos, Especial, Extraordinário)
- Para análise (fallback quando texto insuficiente)

Cada categoria principal pode ter uma lista de subcategorias específicas (ex.: em Sentença: Procedente, Improcedente, Parcialmente Procedente, Homologação de Transação, Homologação de Renúncia). A taxonomia é código-de-verdade em `app/services/classifier/taxonomy.py` e pode ser estendida sem migração de schema.

## Apêndice D — Referências técnicas internas

| Arquivo / recurso                           | Descrição                                                         |
|---------------------------------------------|-------------------------------------------------------------------|
| README.md                                   | Visão geral de execução local e estrutura do repositório          |
| docs/arquitetura_inteligencia_processual.md | Documento mestre do módulo de monitoramento processual            |
| DEPLOY_COOLIFY.md                           | Guia passo-a-passo de deploy em Coolify                           |
| CLASSIFICATION_STATUS.md                    | Status e instrumentação do pipeline de classificação              |
| app/api/v1/endpoints/                       | Handlers de todos os endpoints REST                               |
| app/services/                               | Services de negócio (classifier, batch, monitoring, treatment...) |
| app/runners/legalone/                       | Scripts Node para RPAs específicos no Legal One                   |
| alembic/versions/                           | Histórico de migrações de banco                                   |
| frontend/src/pages/                         | Páginas da SPA (uma por fluxo)                                    |
| prometheus.yml                              | Configuração de scraping de métricas                              |

— Fim do documento —

DunaFlow · by DUNATECH · MDR Advocacia · Abril/2026